Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

3D Platformer Game By Ethan Birks

Contents:

- **Analysis 3**
  - Problem Identification **3**
  - How I can use computational methods to solve the problem **3**
  - In Depth Research **4**
  - Suitable Stakeholders **6**
  - Stakeholder investigation **7**
  - Investigation results **7**
  - Investigation analysis **8**
  - Success criteria **8**
  - Limitations **10**
  - Software Requirements **10**
  - Hardware Requirements **11**
- **Design 12**
  - Decomposition **12**
  - Usability Features **14**
  - How the world is generated **14**
  - How each world will be put together **17**
  - Collisions **18**
  - Player Movement **19**
  - Level Completion **20**
  - Win Message **20**
  - Start message **22**
  - Timer **22**
  - Checkpoint and Death **24**
  - Items **25**
  - Test Plan **25**
  - Development methodologies **27**
  - Story **27**
- **Implementation 29**
  - **Iteration 1 29**
    - Stage One: creating the game loop and rendering a textured cube **29**
    - TEST 1 **33**
    - Stage Two: Camera and Player Class Development **34**
    - TEST 2 **39**

- Stage Three: The Pause Menu **41**
- Stage Four: The timer **45**
- TEST 3 **47**
- Stakeholder Feedback_1 **48**

- **Iteration 2 49**
  - Phase One: The Ground **49**
  - Phase Two: The Platforms **50**
  - Phase Three: My Player **51**
  - TEST 4 **53**
  - Phase Four: The Timer **57**
  - Phase Five: The Pause Menu **59**
  - Phase Six: The Main Game Loop **60**
  - TEST 5 **62**
  - Stakeholder Feedback_2 **63**
- **Iteration 3 64**
  - Phase One: The walls **64**
  - Phase Two: The Camera **65**
  - Phase Three: The Platforms(again) **67**
  - TEST 6 **72**
  - Phase Four draw_2d_text **74**
  - Phase Five Part One: The finish **75**
  - Phase Five Part Two: The Database **76**
  - Phase Six: handle_events **79**
  - TEST 7 **80**

- **Evaluation 82**
  - Final Testing and Success Criteria **82**
    - Final Test **87**
  - Usability features **92**
  - Maintenance **92**
  - Limitations and Improvements **92**
  - Reflection **93**
- **My Code 95**

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Analysis section

Problem Identification

Using Pygame I want to create a new 3D-platformer. Many platformers that are 3D have been made before, but I want to add more challenging elements to the genre. Many games just have you follow a streamlined path from start to finish, some may have shortcuts but may be unintentionally added. Instead of that, using Pyscripter I want to redesign 3D platformers to have more challenging elements including puzzles and items needed to get to the finish. Many games will also have a timer but they only record your times and don't compare and make a leaderboard out of them. I want to use Pygame and a database to create a seamless leaderboard system for the players

How I can use computational methods to solve the problem

When looking at the game I want to make, first I will need to abstract the problem. When creating the game I will need to first remove any type of story for this game. A telling game would need a good story, however with the original concept I build, I will take out the story to be able to design areas for traps, puzzles and challenges. It will help me see where they will best fit when designing the levels on paper and also when making the game. If I try to add texture or add story elements to the game too early it will make the game design harder.

I will also use decomposition to break apart the problems into smaller problems to solve step by step. By doing this I can plan how I will make the game better and split it up into sections when in the development sector. I will focus on separating things like movement level design and other features. By doing this I can make it easier to develop the game and build a solid foundation by making sure I can follow a plan.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Without computational methods it would be hard to make a functioning game, including checkpoints, as they will abstract information from the game they will ignore character design, instead it will focus on game data. Each time a player logs out the game the game will save their player data so any progress they have made to that level will be saved, this will also happen for all three lives a player is given, but instead saves the entire game state so it can reset after death.

To make my game using computational methods I will employ the use of Algorithms, This includes flow charts. I will use the flowchart to plan out how each section will work and be put together into one bigger project. It will help show each area I will code later on and how each section of the game interacts with each other. To give a rough outline of how the code for each section will work I will also implement the use of pseudocode to show how I plan to write the code and fit it all together.

In Depth Research

A new game called Poi released as early access on steam

Poi is a 3d platformer released on 1st February 2017, it is similar to Super Mario Odyssey in that there is a world to explore giving the player the option to adventure around the world to discover hidden areas.

There is a timer, every time a user enters a level that lets them compete against other users to try and get the best time.
The player gets a total of 3 lives to complete the entire level. If the player dies they then can go back to the last saved checkpoint unless they lose all 3 lives then the player is reset back to the start of the level.
Throughout the player's journey they can collect items needed to be able to declare the game as finished.
The player is allowed movement in all directions including diagonals; the movement flows seamlessly between each direction and the character doesn't have sharp movements. The character has the ability to jump and double jump in the air to reach further distances
Poi was made in unity which is coded in mostly C#

Koa and the Five Pirates of Mara

This game has aspects of Poi but also some new features.
This game has a limited amount of levels and areas to explore, with 8 worlds in total, however each world is long and has more challenges put in to make up for this.
The player once again has to collect items to move between the levels.
This game has more of a story element which could be implemented into my own project.
In this game there are enemies to kill and bosses to defeat to unlock area’s or to get items.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

This game was designed to be played on nintendo switch and nothing else at release. This was made in unreal Engine which uses C++ unlike my game that was made on python and pygame

A modelled concept for a new 3d platformer

When looking at Poi, it has some features that I would like to implement like a timer that lets the players compete with each other. However my game will have two timers that monitor overall progress and level progress. I will also implement more movement ideas like a roll or a dash while including already done elements like double jump. This is also the same for Koa and the Five Pirates of Mara. Unlike this game, which has a really short amount of levels implemented, I want to expand on this and include a lot more levels to provide more variety, to ensure the user does not get bored. But I will also try and implement a bit of a story into the game

Suitable stakeholders

My target audience will be students at my sixth form, they will help me decide on features to keep and features that are not needed. They will give me feedback on features to add and how to improve the game. I have selected these people due to their age range and their love for games. The typical age range for gamers is between 14 to 25, so by talking to members of my sixth form I have taken up the

middle ground of the players. I will also reach out to gamers who have played this type of game before and get their feedback on it.

Stakeholder investigation

I will be questioning members of my sixth form, students who range from 16-18 years of age to see their response, These are the questions that i will ask:

- Have you ever played a 3D platformer before?
- If so can they be named?
- What features were good?
- What features were bad?

Investigation Results

I posed these questions to the members of my stakeholder group and tallied their responses up and put them into a suitable and easy to understand order. These students will act as the stakeholders for my project and will give constructive feedback throughout the development of my 3D platformer. Their names are: Lewis Grant, Sean Fraser, Lillie Scott, Aria Robertson, Elliot Mills, Haiden Davidson, Melody Kennedy and Ryan Griffiths

Have you ever played a 3D platformer before?

- Yes (8)

This means that all stakeholders have played some sort of 3D platformer game so they will have experience and all have an opinion on what they would consider is a good game.

If so can they be named?

- Poi (5)
- It Takes Two (4)
- Koa and the Five Pirates of Mara (3)

This gives a good spread of games that have been played by my stakeholders so they will all have different ideas and features they would like to be added.

What features were good?

- Level exploration (6)

- Hidden areas (3)
- Puzzles (2)
- Competitive aspects (4)

These are all aspects that will be tried to be integrated into the game, some may be harder than others.

What features were bad?

- Time limits (5)
- Too complicated (3)
- Messy level design (4)

Some of this will be hard to avoid but to satisfy my stakeholders I will try and avoid having features like these.

Investigation Analysis

Each person I questioned, all have played some form of a 3D platformer, each game they played is different from each other and all have separate features that made them unique.
This means some suggested features may not go well with the overall idea and concept of the platformer I want to create. There will be a mix of the features put together to try and satisfy all the stakeholders.
A compatible feature that would go well is hidden areas and puzzles as they can go hand in hand to meet the level exploration feature, but it may conflict with the complication of level design, so it would have to be moderated.
The features added will be mostly based on the majority answered but I will also put features in to satisfy the other stakeholders.

The results have given me basic success criteria to follow when I get to the design and development phase of the project, here are the requirements:

Success Criteria

Success Criteria	**How it will be measured**
The game will be fullscreen	When the game starts it should boot up in fullscreen mode and cover the entire display it has been given. There will also be code showing this happening.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Success Criteria	**How it will be measured**
The timer will only start when the player starts to move	This should be visible on the screen as when the first step is taken the timer should start. This can be repeated to test for errors. In the code there should be lines saying for this to happen.
The timer does not stop when the player dies	The timer should keep running to prove this and can be tested through multiple different deaths and through code written to ensure this takes place.
A character who can move in all directions - up, down, left, right and diagonally.	Within the code there will be lines that describe how the character will move and the buttons that are needed to be pressed to achieve this effect.
There will be a leaderboard that can track the times recorded of the player that can be compared to other players.	In the code there will be functions that write to a database that will store all the data on times and player info. Each player can see each other's levels times. They should also be able to see all their other times and compare them.
The game will have a multitude of levels to play on	This will be seen as the player progresses through the levels and should encounter no problems moving between each level.
Puzzles and traps on each level	There will be code to describe and give function to puzzles and traps.
Each level will have a different theme	Each level should look different when looking at the graphics for the game and should be clearly visible, such as a desert level ~~and~~ or a snow level.
Tools and items that can be used to navigate each world	Each item and tool will have lines of code assigned to it to provide its function and this should be clearly visible.
There will be two timers one per level and one for all levels combined	This will mean the implementation of two different timers in the code and should be separate. Including a specific timer for each level.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Success Criteria	How it will be measured
On Death when the player dies reset at spawn point or checkpoint, if that checkpoint has been reached	This will use the save checkpoint for when the player stands on one to save a position and keep that until its needed

Limitations

With the way I want to create my 3D platformer using Python and Pygame an obvious limitation appears. The game will only be able to run on computers or devices that have Python and the Pygame plugin installed onto them. Python is also a resource intensive programming language that will use up more resources than most other languages. This is due to Python being an interpreted language not a compiled language. An interpreted language converts the code into machine code in real time. This means that it has to understand and execute code one line at a time while a compiled code is converted into machine code when the code is executed all at once to save resources when the program runs.

A massive limitation for me is the time limit. There are many more features I would want to add to my game however I am operating on a strict time limit so many ideas cannot be implemented and the game may not be finished with all levels before the time is up.

Software requirements

- Python - To run a python game you will need to have python installed onto the computer you plan to play on, which is Python 3.12.0. To best run the game the most updated version of Python will be recommended and the version has to be above Python 2 as Pygame dictates that a version higher than that is needed.

- PIP - This is an installer used the installed Pygame into python, It can be installed from Python.org but usually comes with the version of Python you are running

- Pygame - Pygame is a library that can be installed into python to make aspects of the code easier to make for my game. The most up to date version is recommend as it is necessary to run the game.

- Operating system - Python, can mostly be run on Windows or MacOS, There is some support for Linux but not as much, Most people will be using Windows or Mac so I will limit my comments to those
    - Windows - Python and Pygame will run on any version including and above Windows 7, Python has different versions of the program depending on type of processor you have, as there is 64 bit, 32 bit and ARM64 processor.

    - MacOS has less requirements and doesn't seem to have any limitation concerning what version of MacOS you use and neither does Pygame, however it only has an 64bit installation.

Hardware requirements

There are peripherals that are needed to run this game that are essential:

- The first is a keyboard: I have a 100% keyboard and this is what the game will be based on but the controls will be able to be used on 60% as well. The Keyboard will be in the QWERTY organisation.

- The Second is a mouse: Any type of mouse should work as long as it allows you to move a cursor around the screen. A trackpad or any other type of device that allows you to move the cursor should be fine.

- The final is a Monitor: Any monitor above the size of 720p should be fine as I will try to make the game different for each monitor but they need to be a basic resolution.

- Computer Specs these are what are recommended for Python versions now:
    - CPU: Quad-core processor with 3.0 GHz or above. RAM: 16 GB or more (32 GB recommended for very large datasets) Storage: 256 GB SSD or better (additional external storage is preferred in case large datasets need to be accessed) GPU: NVIDIA GTX 1050 or higher (RTX 2060 for deep learning)

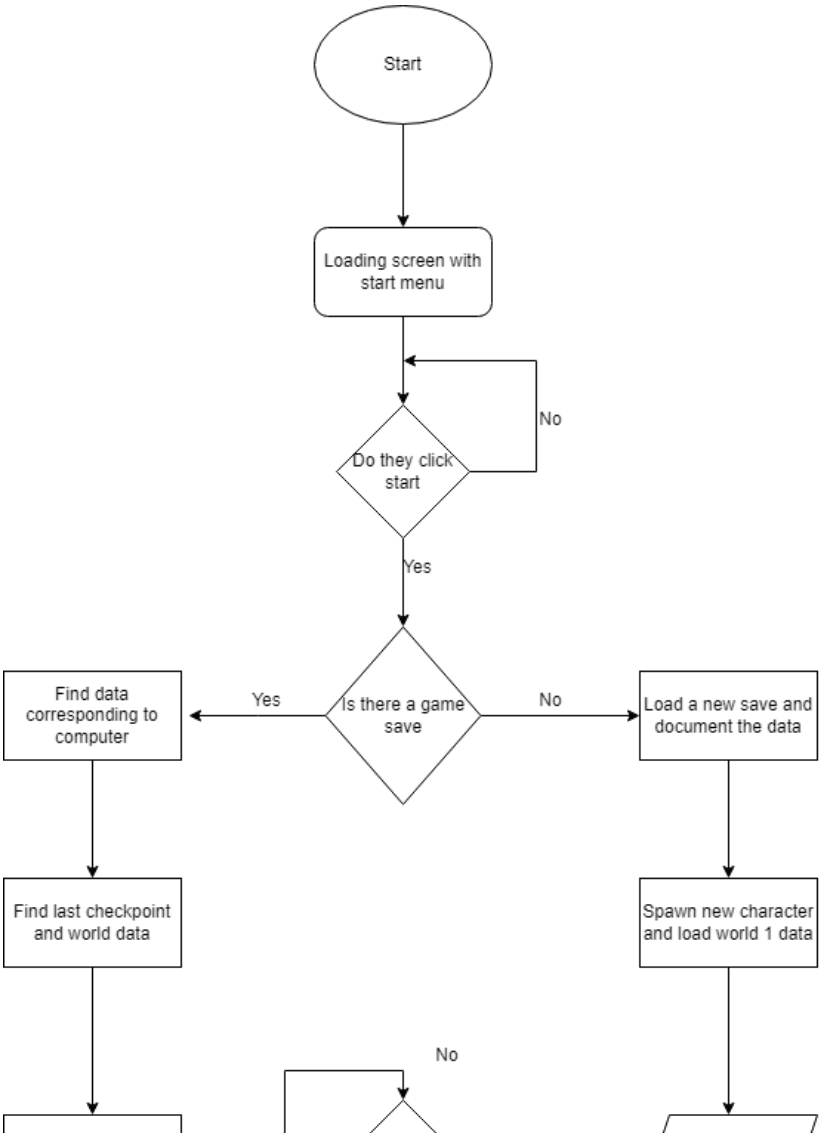
- These are my specs that I am using:
    - I7-2700k 3.4ghz quad core
    - 1tb HDD and 128 SSD
    - Nvidia RTX 3060Ti

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

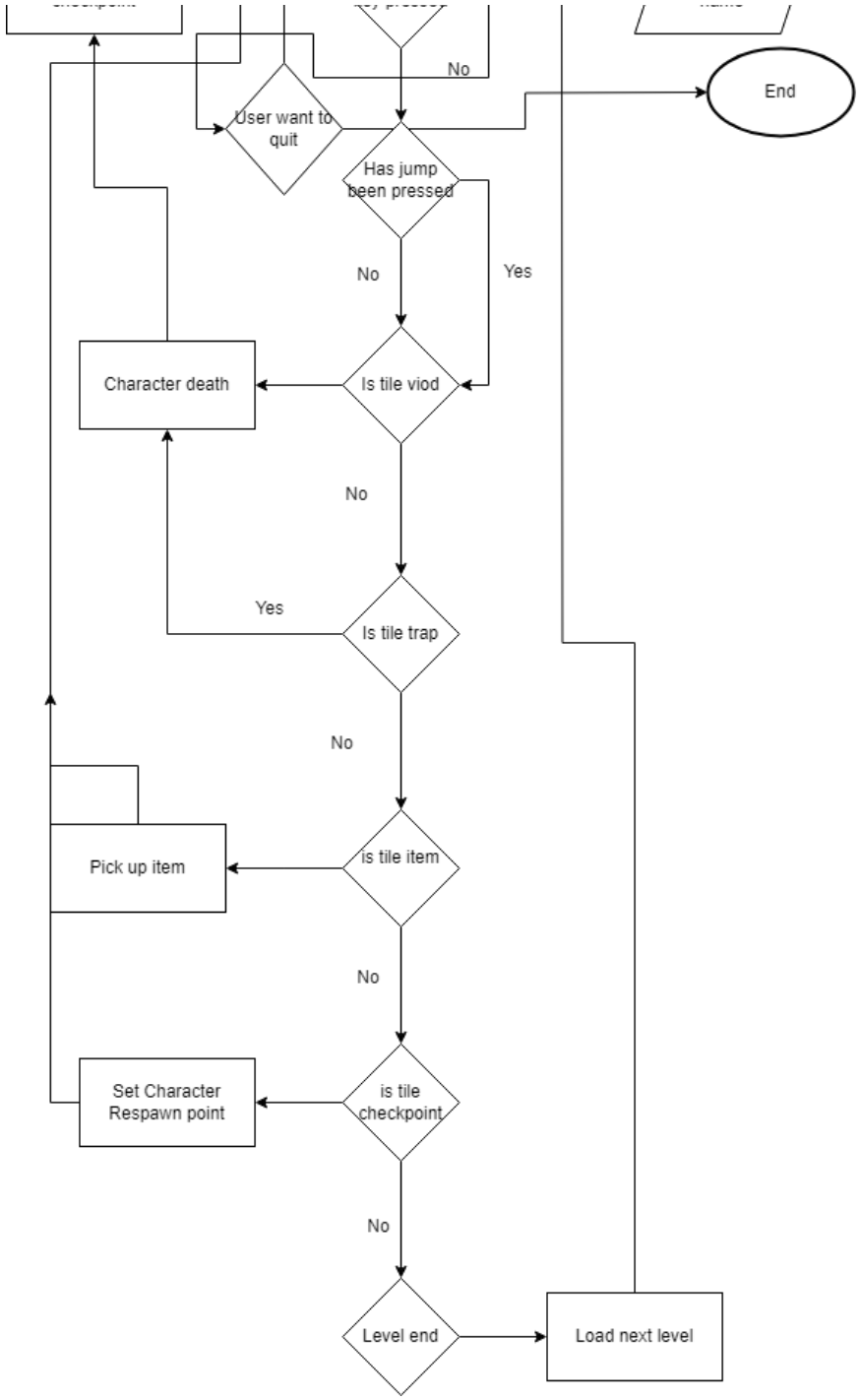
Design Section

Decomposition

Below is a flowchart that I have made that shows how the sub processes will fit together to make my program work



Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework



Within this program there is one big loop that checks if an action is pressed and reacts to it each frame to decide on what to do next. It repeats until the user wants to quit

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Usability Features

A game like the one I'm creating should, in my opinion, have straightforward controls and an easy to understand objective, but it should also be complex enough to require players to uncover hidden regions and solve difficult challenges
I want the game to be a challenge but be simple. That's why I will be using a control scheme very similar to the Poi control scheme. If I do this the game will be easy to understand with the basic movement controls implemented within the game while having the challenging elements come later on.
Pressing the W, S, A and D buttons will move the character in all four cardinal directions and also diagonal lines. This will help my users navigate the game easier. Most games that are made use these four buttons to move a character and have become a sort of standard. As my stakeholders have played video games these will make the most sense to use as they will be familiar with them. Poi also makes use of some jumping mechanics. They have a simple jump and double jump mechanic where the character jumps in all directions while also being able to double jump. This will all make my game more usable to the end user.

The start and exit menu will be interacted with by the W and S keys. I would love to use my cursor to do this but I will run out of time and will not have a functional menu to access the game, so to save some time I will be cutting down on this and using the keyboard keys instead.

How the world is generated

To do this I will use the python PyOpenGL to generate the 3d objects that will make up the levels of my game

- Game will be made up of cubes
- Joined together on same vertices
- Collision settings for each block

If I want to generate a cube in PyOpenGL I will need to label all vertices first to designate how the cube will be generated.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
def get_vao(self):
    vao = self.ctx.vertex_array(self.shader_program, [(self.vbo, '3f', 'in_position')])
    return vao

def get_vertex_data(self):
    vertices = [(-1, -1, 1), (1, -1, 1), (1, 1, 1), (-1, 1, 1),
                (-1, 1, -1), (-1, -1, -1), (1, -1, -1), (1, 1, -1)]

    indices = [(0, 2, 3), (0, 1, 2),
               (1, 7, 2), (1, 6, 7),
               (6, 5, 4), (4, 7, 6),
               (3, 4, 5), (3, 5, 0),
               (3, 7, 4), (3, 2, 7),
               (0, 6, 1), (0, 5, 6)]

    vertex_data = self.get_data(vertices, indices)
    return vertex_data

@staticmethod
def get_data(vertices, indices):
    data = [vertices[ind] for triangle in indices for ind in triangle]
    return np.array(data, dtype='f4')

def get_vbo(self):
    vertex_data = self.get_vertex_data()
```

This code generates the vertices and indices of the cube and links them together

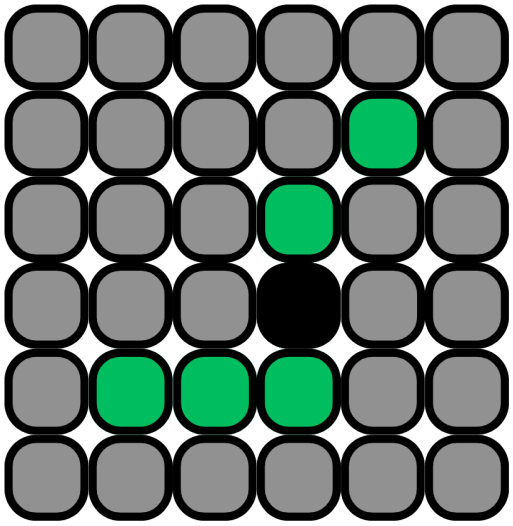
This section of code is used to label the corners of a cube and set the perimeters for the 3d objects to be generated. While this can generate a cube right now there is no way to position the cube on a tile map or generate more than one cube and align them. This only allows single corner generation. I will need to create a grid index of the map for cubes to be generated.

To do this I will utilise an array setup. By using an array I can set up the grid how I want to and design where each object will sit in the array where the video spaces will be generated. I can stack arrays together to build a level. I can also use this to section the game and only make the array that is looked at by the camera to be loaded to save processing time and power on the Computer.

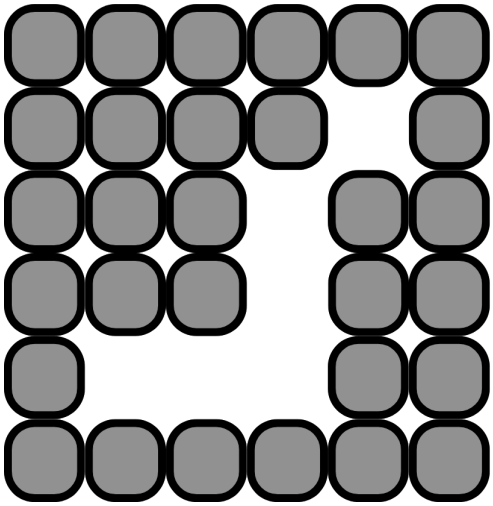
When looking at a 3d plain you have 3 vectors to work on x, y and z these will control all the positions of each individual cube per world

To represent the arrays within docs I have used canva and colour coded squares to mean the different functions in my game:
Gray=wall, Black=void, green=platform

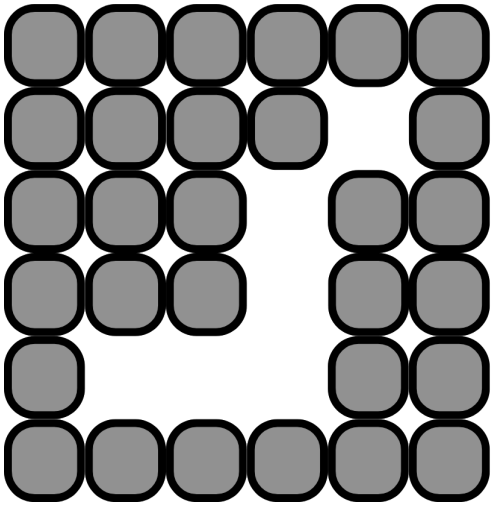
Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework



Array level one with green being floor tiles, grey being wall, and black being void



Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework



These second two will stack on top of each other to build a wall
the player can't jump over

I will overlap these arrays by using the dimensions of the cube to calculate where to position each area so they overlap at the right height and provide a seamless level design.

How each world will be put together

The 3d plane has an X, Y and Z axis that the objects can move through in the 3d plane. By doing this I can align each array set to build a world one step at a time. This will take up a large proportion of processing time as the amount of 3d objects it will need to render will be a lot, however this seems the best way to build something like I am with the limited amount of time.

There could be a way to procedurally generate each world to make the design more random and not have as much human error. However, I will not be doing this due to time constraints and. I think I can better design the level to insert difficulties where I want them rather than having it generated in places and areas that can't be accessed

Instead I can make sure that each level is designed properly and each grid of cubes is aligned to make a seamless design.

This allows me to rotate and position each cube or array of cubes in any position I want to allow the grid of arrays to be put together to allow a level to be made from it.

```
def paintGL(self):
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    glBegin(GL_LINES)

    glColor3d(1, 0, 0)
    glVertex3d(0, 0, 0)
    glVertex3d(1, 0, 0)

    glColor3d(0, 1, 0)
    glVertex3d(0, 0, 0)
    glVertex3d(0, 1, 0)

    glColor3d(0, 0, 1)
    glVertex3d(0, 0, 0)
    glVertex3d(0, 0, 1)

    glEnd()
    # glLoadIdentity()

    glTranslate(self._x, self._y, self._z)
    glRotate(self._ry, 0, 1, 0)
    glRotate(self._rz, 0, 0, 1)

    glEnable(GL_BLEND)
    glBlendFunc(GL_SRC_ALPHA, GL_DST_COLOR)
    glBegin(GL_QUADS)
    glColor4fv((0, 1, 0, 0.6))
    for vertex in range(4):
        glVertex3fv(self.vertices_vertical[vertex])
    glColor4fv((1, 0, 0, 0.6))
    for vertex in range(4):
        glVertex3fv(self.vertices_horizontal[vertex])
    glEnd()
```

Collisions

To make any type of platformer game you need collisions. These will allow objects to not be rendered as something that can be walked through and instead walked on. By applying collisions I can make blocks that can be walked on and interacted with. These are essential when generating the cubes and the world as a whole.

I will do this by making use of the Check Collision Function. This will allow me to apply collisions easily. By using an x, y, and z that PyOpenGL gives me to designate all the corners of a cube. If two places are overlapping on coordinates it will return true for collisions. When returning a true a will make sure the character is affected by this. If it is on a normal tile they will stay at that level on X,Y,Z. If it is a trap that is detected, the player should die. This is how i will check for collisions within my 3d platformer

Player Movement

To play a 3d platformer you need to be able to move a character around a plane. I will do this by making sure a certain key is pressed will lead to a change in the axis of the character.

```
for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_LEFT:
                    glTranslatef(-0.5,0,0)
                if event.key == pygame.K_RIGHT:
                    glTranslatef(0.5,0,0)

                if event.key == pygame.K_UP:
                    glTranslatef(0,1,0)
                if event.key == pygame.K_DOWN:
                    glTranslatef(0,-1,0)
```

This code allows me to change the position of a cube/character on the grid and position with the press of a key. With each key pressed it will alter the position of the character by 0.5 on the axis either way left or right. While I can also alter it by 1 on the Y axis for when the jump key is pressed.

I can also check, while doing this, if the character is colliding with another object on the world map by using the Check Collision function to monitor every step if the character collides with another object or if there is now a void tile in its space. This will all allow my

character to traverse the 3d plane properly without problem with colluding with a platform and be able to perform the basic movement actions i want it to.

Level Completion

I need to design a function that will allow the user to know when they have completed a level and check if the level has actually been completed. This means that an item that is needed has been picked up and also placed on the right tile to complete this game. This can be done by using a win detection function.

```
function checkWin ():
     While [x,y,z] != [1,1,1]
          won = False
     won = True
     Return won
```

When designing this algorithm I have labelled the character as a shape as while playing you cannot see the character you are playing. This small algorithm checks the position of the character and compares it to the tile that is setup as the win condition and if the their X, Y, Z overlap declares that level over.

Win message

The game needs to display that the win condition has been met. To do this I need my game to display win conditions. This simple algorithm will achieve it.

```
won = False
while won == False:
     world1()
     Timer = updateTimer()
     won = checkWin()
winMessage()
```

Timer represents the timer that will be running as you run through the game. ~~and~~

world1() represents the whole procedure needed to run the game and load all the necessary items needed to load the game.

The algorithm infinitely loops until the win condition is met.

I want the timer displayed while also showing the win message and how to move onto the next level. Here is the algorithm to do this.

```
fill screen (0, 0, 0)
draw text(timer, size, (timeX, timeY))
draw text("Level Complete", > size, (windowX/2, wonY))
draw text ("Press enter to enter the next level. ", size,
(windowX/2, < wonY))
```

The first line will fill the entire screen white to show that the game has stopped and indicate something will change on the screen. As I am not certain where on the screen I will want the timer to sit, I have decided not to specify as right now that can be changed during development. I have designed the third line to display a big ‘Level Complete’ message in the middle of the screen to show that the level is over. Finally my last line will make sure they know what to do next to enter the next level and I will do this by using the enter key later on when coding.

Now I also need to show how this will work:

```
continue = False
while not continue:
     events = getEvents
     for event in events:
          if event == enterKeyDown:
               continue = True
               loadNextWorld()
```

These all come together to make a function win message and the ability to move forward onto the next world. This shows the algorithms to do this.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Start Message

```
fill screen (0, 0, 0)
draw text(timer, size, (timeX, timeY))
draw text("Welcome", > size, (windowX/2, wonY))
draw text ("Press enter to start. ", size, (windowX/2, < wonY))
```

This function will operate in an very similar way to the level completion screen however instead it will boot the character using the enter key into a new world and start the process again from the beginning.

Timer

Timer is one of the more simple aspects to code for my NEA as there is a pygame object for pygame.time.clock. This can be used to keep track of time within the program and there are two aspects needed to do this. pygame.time.clock.tick(), pygame.time.clock.get_time().

The issue with this is that pygame.time.clock displays time in milliseconds and I don't want that for my game as I want to get my timings written for now. This does mean that now I have to create a function to remove this feature.

```
function millisecondConverter (milliseconds):
     seconds = (milliseconds/1000)%60
     minutes = (milliseconds/(1000 * 60))%60
     hours = (milliseconds/(1000 * 60 * 60))
     time = (str(hours) + ": " + str(minutes) + ": " +
str(seconds))
     return time
```

All this function does is convert the time from milliseconds, so now I need to show the timer and get it to time the game properly.

```
while (not won) and (not quit):
     time += clock. get_time()
     world()
     time = updateTime()
     won = checkWin()
```

```
    clock. tick(60)
```

This will tick the time of the timer over, while also updating the timer on the screen to keep an accurate account of the time that the user is taking to complete the world that they are in. By doing what I am doing with this timer the game will be limited to a framerate of 60fps but that is fine as most computers will be able to run the game reliably at the speed.

I will be tracking the time of each user and entering them into a public database. To do this I will need validation. To make sure that the database knows which character the timer is coming from.

```
from requests import get
ip = get('https://api.ipify.org').content.decode('utf8')
ipValidation = format(ip)
While nameInput != nameStored AND passInput != passStored
    nameInput = (str(Input)
    passInput = (str(input))
```

This uses an already made tool that can grab the external IP address of the user and then input that into a variable ipValidation which will then be used to check the database if it exists already. To fix a issue where only one ip address is allowed locking it to one account per household. I would use a composite key in my database to allow multiple users per ip address as long as they have unique username and passwords.

```
try:
    with conn.cursor() as cursor:
        # Update a record
        sql = "UPDATE `users` SET `timer`=%s WHERE `ip`=%s"
        cursor.execute(sql, ('timer', 'ipValidation'))

    # Commit changes
    conn.commit()

    print("Record updated successfully")
Finally:

    conn.close()
```

```
If sql.ip != ipValidation:
      sql = """INSERT INTO timesheet(
      Users, ip, timer)
      VALUES ('nameInput','ipValidation','timer')"""
      try:
            # Executing the SQL command
            cursor.execute(sql)

            # Commit your changes in the database
            conn.commit()
      except:
            # Rolling back in case of error
            conn.rollback()

           # Closing the connection
           conn.close()
```

This will insert the data and replace the timer of the old run

Checkpoint and Death

```
if tile == checkpoint.tile()
      Save = [X,Y,Z]  = shape[X,Y,Z]
      Save = [X,Y,Z] = Save = [X,Y,Z]
```

These two lines of code will take the XYZ of the character and save them into a new variable called Save. This variable will hold the XYZ location of the last checkpoint touched by the character and hold that until it is needed on the death of the character. The character will respawn at the Save XYZ when death occurs.

```
If tile == void.tile OR tile == trap.tile()
      shape[X,Y,Z] = Save[X,Y,Z]
```

These two lines of code will make it so the character will respawn at the last checkpoint saved. It will also make sure no progress is lost while keeping the timer active as it is loaded under a different function not the character as I am not resetting the entire character structure.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Items

Each item will be generated like my cubes on the 3d plane but I will add a different collision box to them as shown.

```
checkCollision(x,y,z)
If collision == true
        Item = True
```

This will allow items to enter and inventory of the character and become a fulfilment criteria to end the game when it checks if the item is now in the inventory of the character as the tag will be ticked.

Test Plan

Test	Input	Expected Result
Load Program	Click to run p	Game screen covers the entire monitor making it fullscreen The loading screen with the start button will appear
Exit program	The generated X button on the window is pressed or button on game menu	cancel the program and shut it down
Start Button pressed	On the menu the start button pressed	The game will load you into the first world and load all items and setup timer
Pause menu	Esc pressed	This will bring up the menu that can allow the user to exit the game
World generated	When start is pressed load up the world generation	When start is pressed the code should then head to the world1() function and load this into the game state

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Test	Input	Expected Result
Character generation	After start pressed and worldGen() functions runs	The character will then be initialised from the function character() and should allow a camera attached to the character to be moved
Look up	Move mouse up	The characters camera should move to look upwards until it reaches the designated max angle
Look Down	Move mouse down	The characters camera should move to look downwards until it reaches the designated max angle
Walk forward	W key on keyboard pressed	Moves the player forward at constant rate Start the timer if not already started
Walk Left	A key on keyboard pressed	Move the player left at a constant rate Start the timer if not already started
Walk Backwards	S key on keyboard pressed	Move the player backwards at a constant rate Start the timer if not already started
Walk Right	D key on keyboard pressed	Move the player right at a constant rate Start the timer if not already started
Jump	The space bar on keyboard is pressed	Cause the player to move upwards Could be vertically or follow the direction of movement key pressed
Double Jump	The space bar on key board is pressed twice	Cause the player to jump twice, one time from the floor other from the air when pressed in either vertical or in direction key pressed
Wall collisions	Player and wall collide	When pressing the W,A,S,D or jump when the player collides with a wall will it stop
Platform collisions	Player and platforms collide	When pressing the W,A,S,D or jump when the player collides with a platform will it stop
Check Point	W,A,S,D pressed until on tile	This should utilise the checkPoint() function to save the current X,Y,Z pos of the character to be used upon character death
Player death	W,A,S,D Or Jump pressed	Character resets to last checkpoint Timer stays constant and doesn't stop
Timer save	PortalCheck() runs after going over portal tile with W,A,S,D	At the end of a level the timer should save the data to a database There should be the option to compare times

Test	Input	Expected Result
Portal check	W,A,S,D pressed until on tile	Each level has an exit portal 7 generated in total Each such take you to next level End one to winning zone
Traps	W,A,S,D pressed until on tile	Each level has different traps on them Traps kill the player on touch Resets after certain time
Item	W,A,S,D pressed until character has gone over item	Item shown on ground Item disappears if ran over Can be placed on exit zone
Unmapped key pressed	A key not mapped is pressed	This should not affect anything to do with game and when pressing any of the keys nothing should happen and the game will continue to run
Quitting	When a key is pressed to quit the game “Q”	The game stops what its doing and saves the data of the run on the account to be rebooted when the game next loads

Development methodologies

When developing my 3d Platformer I will be using the System life cycle to develop my game. It involves analysis, design, development, testing and maintenance. This allows me to analyse different platformer games and then design my own solution to the problem I want to solve. I can then code my game and test it all and make sure each feature works as intended. This means that we can loop infinitely until I get a project that I am happy with.

There are other development methodologies that exist like the waterfall method. As you don’t look back and go over what you did and try to make your design, which is based on a detailed analysis section that has been thorough in its research. you follow one step at a time and finish with maintenance and look and where mistakes were made and write a report on that. That is where a method like the spiral method works where at the end of each system life cycle you would give a prototype to your stakeholders and they would give you the feedback to improve and change details within the code.

Other methods include Agile, which is very similar to Spiral where you develop pieces of code independently and slowly increase the functionality by developing,

test, refine, improve cycle where each function can be developed independently of other functions to make the final game.

Story

To make a proper 3d platformer there needs to be a story element pushing the reason for going through each level. So I have had to come up with this story.

“There has been a war that caused their home world to fall into chaos. To stop this ‘your character’ goes on adventure to find six artefacts hidden over 6 different worlds. Without these the world will collapse and billions of humans will die. You have to locate these, travel over hazardous terrains, reach the planet's core, and then toss them in. Billions of lives will be saved, and the world will stabilise.”

This is the end goal to achieve when setting up the story and I will of course have to subtly implement this into the game with NPCs and storyboards

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Implementation

Stage one: creating the game loop and rendering a textured Cube

To me, when creating a 3d platformer like this, I first need an area for the character to walk on. This to me is more important than creating a character class first. To do this I wrote a basic python program to achieve this

Since this game utilises pygame, PyOpenGL, Numpy and Math libraries I wrote a few lines to allow these libraries to be accessed by my code

```
1 import pygame
2 from pygame.locals import *
3 from OpenGL.GL import *
4 from OpenGL.GLU import *
5 import numpy as np
6 import math
```

This is all quite self explanatory: the first line initialises the pygame library, second, third and fourth for PyOpenGL, fifth numpy and sixth for the Math library.

Next step is to actually create a textured cube, to do this on a 3d plain you need 3 variables to help you. Vertices, faces and tex_coords

```
8  # Define vertices and faces for the cube, using coordinate tuples for each vertex.
9  # `vertices` represent corner points of a cube in 3D space.
10 vertices = (
11     (1, -1, -1),
12     (1, 1, -1),
13     (-1, 1, -1),
14     (-1, -1, -1),
15     (1, -1, 1),
16     (1, 1, 1),
17     (-1, -1, 1),
18     (-1, 1, 1)
19 )
20
21 # Define faces of the cube by grouping indices from `vertices`.
22 faces = (
23     (0, 1, 5, 4),
24     (1, 2, 7, 5),
25     (2, 3, 6, 7),
26     (3, 0, 4, 6),
27     (4, 5, 7, 6),
28     (0, 1, 2, 3)
29 )
30
31 # Define texture coordinates for mapping an image to each cube face.
32 tex_coords = (
33     (0, 0),
34     (1, 0),
35     (1, 1),
36     (0, 1)
37 )
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Line 10 initialise a tuple for each vertex in the cube that can be used to identify it's position on the 3d space

Line 22 Now takes each vertex and group them in a way that allows faces to form on the cube that can be edited

Line 32 defines the coordinates for the program to map the texture to the cube's faces

Next I need to create a function that can access these variables and use them to make a cube.

```
62 # Draw a textured cube based on the `faces` and `vertices` definitions above.
63 def Cube(texture):
64     glBindTexture(GL_TEXTURE_2D, texture)
65     glBegin(GL_QUADS)
66     for face in faces:
67         for i, vertex in enumerate(face):
68             glTexCoord2fv(tex_coords[i % len(tex_coords)])   # Map texture coordinates to each face.
69             glVertex3fv(vertices[vertex])
70     glEnd()
```

Line 63 defines the function as Cube while also allowing it to access the variable texture

Line 64 binds the texture that has been specified in the variable texture to be binded to the cube

Line 65 marks the start of the drawing of the cubes

Line 66 and 67 loop through each face and then each vertex per face while applying the texture coordinated and defining their position within the 3d space

Line 68 does the texture mapping by cycling through each coordinate defined in the tex_coords and making sure each vertex has a corresponding texture coordinate

Next I need to design a way for a texture to be allowed to be accessed by OpenGL and pygame and be set up to render onto the cube

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
40 def load_texture(texture_path):
41     try:
42         # Load image with pygame, flip it vertically, and convert to OpenGL-readable format.
43         texture_surface = pygame.image.load(texture_path)
44         texture_surface = pygame.transform.flip(texture_surface, False, True)  # Flip the texture vertically
45         texture_data = pygame.image.tostring(texture_surface, "RGB", True)
46         width, height = texture_surface.get_size()
47 
48         # Generate and bind a new OpenGL texture.
49         texture = glGenTextures(1)
50         glBindTexture(GL_TEXTURE_2D, texture)
51 
52         # Configure OpenGL to render the texture.
53         glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, texture_data)
54         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
55         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
56 
57         return texture
58     except Exception as e:
59         print(f"Unable to load texture: {e}")
60         return None
```

Lines 43-46 all work together to take an image that is loaded with the pygame engine and flip it vertically, then convert it so it is readable by OpenGL so it can be applied to an object.

Lines 49-50 generate and bind the new texture.

Lines 53-55 take the OpenGL and make sure it is configured in the right way to make sure the texture can be loaded correctly.

Now we need to insert the actual texture into the code so it can be loaded by our function.

```
322     # Load grass texture
323     texture_path = "grass.png"  # Ensure this file exists in the same directory
324     texture = load_texture(texture_path)
325     if texture is None:
326         print("Failed to load texture. Exiting.")
327         pygame.quit()
328         return
```

Line 323 takes the texture path and saves it to a variable

Line 324 then updates the texture variable to be the function load_texture with the new texture path within it.

The code then tries to load this texture onto the cube and if texture fails to do this the program will close as the game has encountered an error.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Next I need to make a pygame window that accesses OpenGL.

```
300     pygame.init()
301     display = (1920, 1080)
302     pygame.display.set_mode(display, DOUBLEBUF | OPENGL)
303     pygame.mouse.set_visible(False)
```

Line 300 will initialise pygame and make sure we have access to the library for pygame or else all code will not work.

Line 301 sets the display size as I want the game to be fullscreen. I have opted for a 1920 x 1080 resolution as this is a standard for my stakeholders and most gamers.

Line 302 does two things, first set up a DOUBLEBUF, which is where the rendering is done off-screen before being displayed, while it also applies OPENGL to the display to allow 3d objects to be displayed.

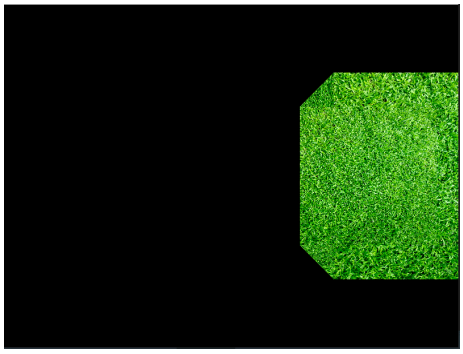
Line 303 makes sure that while playing you can't see the mouse and it doesn't disrupt the core gameplay.

Running this code as it is now, all that would happen is a black screen would appear with no mouse cursor as there is no correlation between the vertices, faces and tex_coords. The pygame window is defaulted to black as nothing has been drawn onto it. It will also not be able to quit through the quit button as that has not been coded into it yet. The screen will be scaled at 1920 x 1080 so the quit button will not actually appear on most monitors due to the fullscreening of the game.

The end result is this

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

TEST 1

Test	Y coor	X coor	Result
Moving the Cube	0	0	
	0	2	
	2	2	

Test	Y coor	X coor	Result
	-1	2	
	0	-1	

Stage Two: Camera and Player Class Development

A 3d Platformer can not operate without a player to move across the level and interact with the world that has been created. I will be using object oriented coding to create the Player as Object Oriented Coding is best for creating a character for a game and the camera needed.

```
126 # Player class for handling movement, position, and gravity.
127 class Player:
128     def __init__(self):
129         self.position = np.array([0, 1, 0], dtype=float)
130         self.velocity = np.array([0, 0, 0], dtype=float)
131         self.size = 0.5
132         self.on_ground = False
```

Line 127 initialises the player class that will hold all the information needed to make the player class work.

Line 129 sets the position of the player when the game is first run to be 1 higher on the Y axis than the cubes generated, it also sets the value as float so it can change on a decimal scale across the X, Y, Z axis.

Line 130 does the same but instead the velocity of the player is set to zero so the player doesn't move till input is registered again. Values on the X, Y, Z axis can all be edited as the player moves.

Line 131 is set up to make the player half the size of the cubes that have been generated already.

Line 132 makes it so the player will use the setup collisions on the cube to fall till that changes to True.

Now we have the basics set up we need to render an actual player so that it can be moved around the screen.

```
134     def draw(self, texture):
135         glPushMatrix()
136         glTranslatef(*self.position)
137         glScalef(self.size, self.size, self.size)
138         draw_textured_cube(texture) # Draw player as a scaled cube.
139         glPopMatrix()
```

Line 136 takes the self position of the player that was defined earlier by accessing the self function and setting that as the position that OpenGL will render the player.

Line 137 again accesses the self function to pull the scale of the cube and applies it to all three of the axes used to generate the cubes.

Line 138 Does what we did to generate the cube before and does the same for our character but just on a smaller scale with a texture still.

The player needs to be able to move around a level this is how I will do this

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
141     def move(self, keys):
142         speed = 0.1
143         jump_strength = 0.125
144         # Handle movement in each direction based on key input.
145         if keys[pygame.K_d]:
146             self.position[0] -= speed
147         if keys[pygame.K_a]:
148             self.position[0] += speed
149         if keys[pygame.K_w]:
150             self.position[2] -= speed
151         if keys[pygame.K_s]:
152             self.position[2] += speed
153         if keys[pygame.K_SPACE] and self.on_ground:
154             self.velocity[1] = jump_strength # Jump.
155             self.on_ground = False
```

Line 142 specifies the speed that player will travel along, this was done at this speed because this was found to be the middle ground among the stakeholders.

Line 143 Again the jump_strength is about how strong the jump is and how long it takes to fall, this was the best match to the stakeholder suggestions.

Lines 145 - 155 makes use of these two variables to make the player move by watching for the keyboard inputs

This is a function that provides the gravity needed to make the player fall back down after each jump.

```
157     def apply_gravity(self):
158         gravity = -0.01
159         if not self.on_ground:
160             self.velocity[1] += gravity
161         self.position += self.velocity
162         if self.position[1] < 0: # Stop falling when reaching the ground.
163             self.position[1] = 0
164             self.on_ground = True
165             self.velocity[1] = 0
```

Line 158 sets the gravity to -0.01 which allows the player to be able to jump and fall without much restriction, while also allowing the player to fall at a constant speed.

Line 160 takes the self.velocity of the player and applies gravity to it so the player starts to fall after the jump.

Lines 162 - 165 takes the self position of the player and looks if it is less than 0 and stops the player falling, while also making the variable self.on_ground true so the player can jump again

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
127 class Player:
128     def __init__(self):
129         self.position = np.array([0, 1, 0], dtype=float)
130         self.velocity = np.array([0, 0, 0], dtype=float)
131         self.size = 0.5
132         self.on_ground = False
133 
134     def draw(self, texture):
135         glPushMatrix()
136         glTranslatef(*self.position)
137         glScalef(self.size, self.size, self.size)
138         draw_textured_cube(texture) # Draw player as a scaled cube.
139         glPopMatrix()
140 
141     def move(self, keys):
142         speed = 0.1
143         jump_strength = 0.125
144         # Handle movement in each direction based on key input.
145         if keys[pygame.K_d]:
146             self.position[0] -= speed
147         if keys[pygame.K_a]:
148             self.position[0] += speed
149         if keys[pygame.K_w]:
150             self.position[2] -= speed
151         if keys[pygame.K_s]:
152             self.position[2] += speed
153         if keys[pygame.K_SPACE] and self.on_ground:
154             self.velocity[1] = jump_strength # Jump.
155             self.on_ground = False
156 
157     def apply_gravity(self):
158         gravity = -0.01
159         if not self.on_ground:
160             self.velocity[1] += gravity
161         self.position += self.velocity
162         if self.position[1] < 0: # Stop falling when reaching the ground.
163             self.position[1] = 0
164             self.on_ground = True
165             self.velocity[1] = 0
166 
```

```
167     def update(self, keys):
168         self.move(keys)
169         self.apply_gravity()
```

The Player class needs a camera attached to it so the user can look around and see the level they have to play; this can be done through a camera class also being created.

```
 93 # Camera class to handle movement, rotation, and view direction.
 94 class Camera:
 95     def __init__(self):
 96         # Initialize position, pitch, and yaw for view orientation.
 97         self.position = np.array([0, 1, -15], dtype=float)
 98         self.pitch = 0
 99         self.yaw = 0
100         self.sensitivity = 0.1 # Mouse sensitivity.
```

The **__init__(self):** function creates the position, pitch and yaw for the camera class to allow movement of the camera by the player by using the self.sensitivity variable which sets the sensitivity for the mouse to move the camera

Now I need a way to update the camera using the sensitivity that was already described above. To do this I do the following

```
102     def update(self, dx, dy):
103         # Update yaw and pitch based on mouse input.
104         self.yaw += dx * self.sensitivity
105         self.pitch -= dy * self.sensitivity
106         self.pitch = max(-89, min(89, self.pitch)) # Limit pitch to avoid flipping.
```

Line 104 and 105 take the pitch and yaw and then multiply the values by the mouse sensitivity, to pan the camera on the pitch and yaw.

Line 106 takes the pitch and sets a limit ~~to it~~ to stop the camera from flipping on itself and makes sure the player can't do a 360 turn on the pitch.

```
108     def apply_view(self, player_position):
109         # Calculate view direction vector.
110         direction = np.array([
111             math.cos(math.radians(self.pitch)) * math.sin(math.radians(self.yaw)),
112             math.sin(math.radians(self.pitch)),
113             -math.cos(math.radians(self.pitch)) * math.cos(math.radians(self.yaw))
114         ])
115         camera_target = player_position + direction
116         up = np.array([0.0, 1.0, 0.0])
117 
118         # Apply view transformation.
119         gluLookAt(
120             self.position[0] + player_position[0], self.position[1] + player_position[1],
121             self.position[2] + player_position[2],
122             camera_target[0], camera_target[1], camera_target[2],
123             up[0], up[1], up[2]
124         )
```

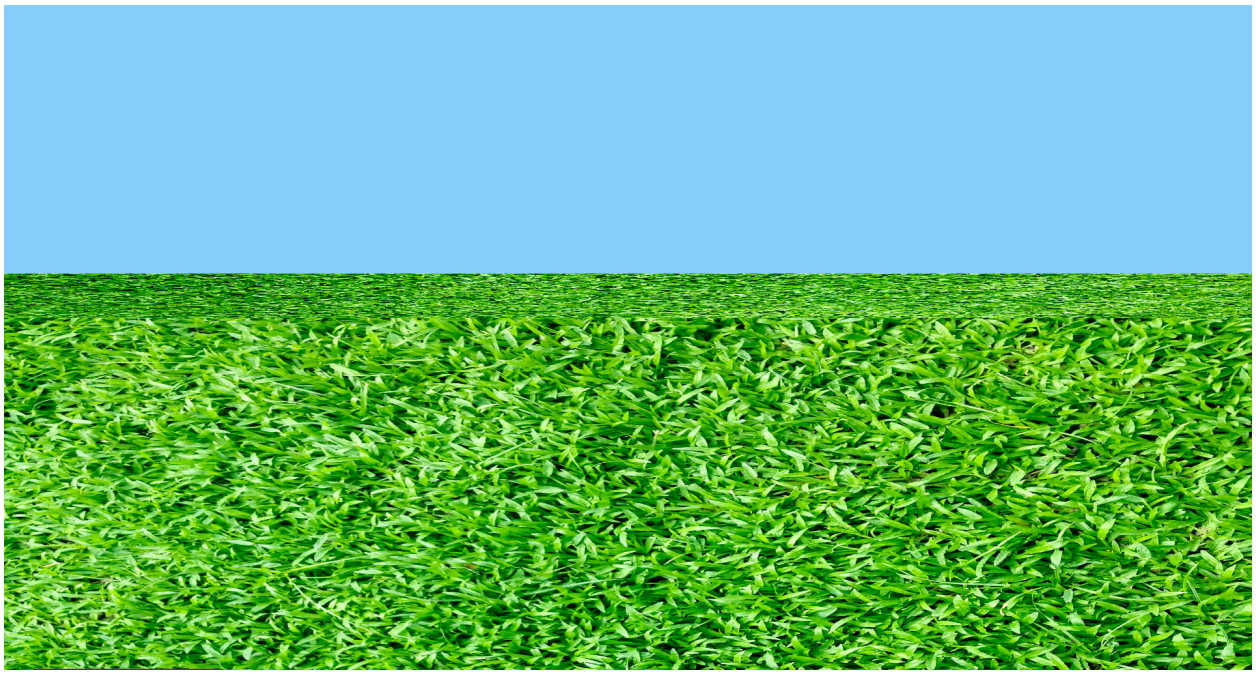
This function all comes together to perform the actions to apply the view of the camera to the player character and the transformation when the player moves the camera by doing maths and using arrays to apply the camera view.
Now there is a player and a camera, I also decided to update the background colour of the game so it is visible to see what is going on and fit the theme of the game. That is why light blue was chosen

```
317     # Set up OpenGL view and background color
318     gluPerspective(45, (display[0] / display[1]), 0.1, 50.0)
319     glClearColor(0.53, 0.81, 0.98, 1)  # Light blue background
```

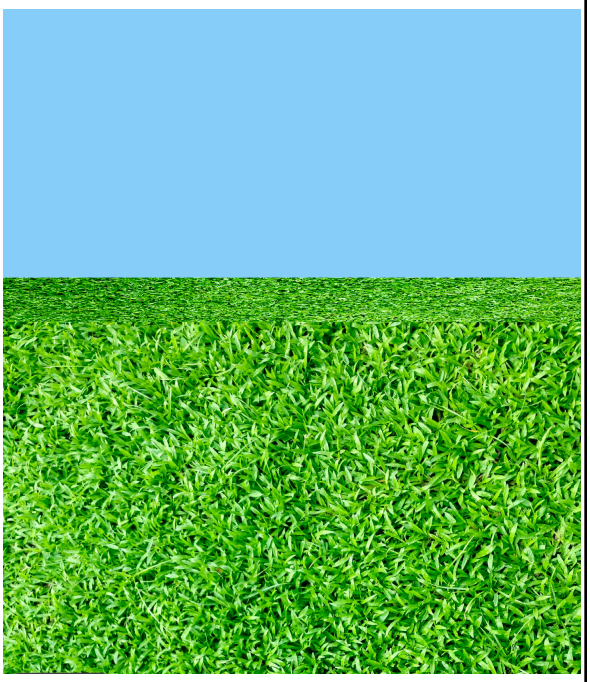
These two lines of code implement this by using the gl colour codes to change the background colour of the game.

It leaves the game looking like this:

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework



TEST 2

test	code	expected	Result
Ground level	ground level = 0	when jumping they will jump and fall back down until Y-level one is reached. And then the fall is stopped and on ground is now true	

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

test	code	expected	Result
Move Left	if keys[pygame.K_a]: self.position[0] += speed	For all move tests the speed will be a constant one	Self.position[0] = 0 Self.position[0] = 1 Self.position[0] = 2 Self.position[0] = 3
Move Right	if keys[pygame.K_d]: self.position[0] -= speed	For all move tests the speed will be a constant one	Self.position[0] = 0 Self.position[0] = -1 Self.position[0] = -2 Self.position[0] = -3
Move Forward	if keys[pygame.K_w]: self.position[2] -= speed	For all move tests the speed will be a constant one	Self.position[2] = 0 Self.position[2] = -1 Self.position[2] = -2 Self.position[2] = -3
Move Backwards	if keys[pygame.K_s]: self.position[2] += speed	For all move tests the speed will be a constant one	Self.position[2] = 0 Self.position[2] = 1 Self.position[2] = 2 Self.position[2] = 3

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Stage Three: The Pause Menu

I want my game to have a pause menu as it will be important as the player may not be able to play continuously, they might need to take a 5 minute break. This will allow the game to be on pause until the player restarts the game

```
171 # Display a semi-transparent pause menu.
172 def display_pause_menu():
173     # Set up a surface for the pause overlay
174     pause_surface = pygame.Surface((1920, 1080), pygame.SRCALPHA)  # Full screen
175     pause_surface.fill((0, 0, 0, 128))  # Semi-transparent black background
176 
177     # Render the pause title
178     font = pygame.font.SysFont("Arial", 56)
179     title_text = font.render("PAUSE MENU", True, (255, 255, 255))
180     title_rect = title_text.get_rect(center=(960, 350))
181     pause_surface.blit(title_text, title_rect)
182 
183     # Render the instructions
184     instructions_font = pygame.font.SysFont("Arial", 24)
185     instructions = [
186         "Game is paused. You have the following options:",
187         "",
188         "- Resume: Press ESC to continue your adventure!",
189         "  Your current progress will be saved and you can pick up right where you left off.",
190         "",
191         "- Exit: Press Q to quit the game.",
192         "  Your progress will not be saved.",
193         "  Are you sure you want to exit?",
194         "",
195         "Use the ESC key to return to the game or Q to exit."
196     ]
```

Line 174 and 175 set the size of the pause screen to the fullscreen of the players monitors as set out by my stakeholders and make the pause screen black. This is a neutral colour voted on by my stakeholders.

Line 178 - 181 work together to make the title of the pause menu in the font arial. This will be subject to change from stakeholders. It renders the text just a bit above the middle of the screen to leave room for the instructions to follow.

Line 184-196 is the instructions put into a list that has separation going down by using empty spaces then a new line allowing the instructions to be presented clearly to the player.

Now I've set out what the title screen will display. Now I need to actually render the text and background.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
198     # Render each line of instructions
199     for i, line in enumerate(instructions):
200         instruction_text = instructions_font.render(line, True, (255, 255, 255))
201         text_rect = instruction_text.get_rect(center=(960, 400 + i * 40))
202         pause_surface.blit(instruction_text, text_rect)
```

These 4 lines of code come together to render each line of the instructions stored in instructions. This will display them under the Pause Menu text that has already been rendered.

At the moment this will only render on a 2d plane not on a 3d one like I wanted it to. I need to fix this or else the menu will never display

```
204     # Convert pause surface to texture and draw it (same as before)
205     # Convert pause surface to texture
206     texture_data = pygame.image.tostring(pause_surface, "RGBA", True)
207     pause_texture = glGenTextures(1)
208     glBindTexture(GL_TEXTURE_2D, pause_texture)
209     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE)
210     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE)
211     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
212     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
213     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, pause_surface.get_width(), pause_surface.get_height(), 0, GL_RGBA, GL_UNSIGNED_BYTE, texture_data)
214
215     # Set up 2D orthographic projection for overlay drawing
216     glMatrixMode(GL_PROJECTION)
217     glPushMatrix()
218     glLoadIdentity()
219     glOrtho(0, 800, 600, 0, -1, 1)
220     glMatrixMode(GL_MODELVIEW)
221     glPushMatrix()
222     glLoadIdentity()
223
224     # Apply the mirror and rotate transformations to the pause menu (as before)
225     glEnable(GL_TEXTURE_2D)
226     glBindTexture(GL_TEXTURE_2D, pause_texture)
227     glColor3f(1, 1, 1)
228
229     glPushMatrix()  # Save the current matrix state
230
231     # Translate to the center of the pause menu
232     glTranslatef(400, 300, 0)  # Center of the screen
233     glRotatef(180, 0, 0, 1)    # Rotate 180 degrees
234     glScalef(-1, 1, 1)         # Mirror horizontally
235     glTranslatef(-400, -300, 0) # Translate back to original position
```

```
237     # Draw the pause menu quad
238     glBegin(GL_QUADS)
239     glTexCoord2f(0, 1)
240     glVertex2f(0, 600)
241     glTexCoord2f(1, 1)
242     glVertex2f(800, 600)
243     glTexCoord2f(1, 0)
244     glVertex2f(800, 0)
245     glTexCoord2f(0, 0)
246     glVertex2f(0, 0)
247     glEnd()
248
249     glPopMatrix()  # Restore the previous matrix state
250     glDisable(GL_TEXTURE_2D)
251
252     # Restore 3D projection
253     glMatrixMode(GL_PROJECTION)
254     glPopMatrix()
255     glMatrixMode(GL_MODELVIEW)
256     glPopMatrix()
```

This all forms the code to allow the pause menu to be displayed on a background with text.

It does this by first taking the pause menu we have generated and then turning all this into a texture that can be displayed on the screen, then the code sets up an area for the Pause menu texture to be applied by using orthographic projection.

Then it sets up the texture on the 2d orthographic projection and binds all that together. The pause menu has to then be rotated 180 degrees and then mirrored to make the text display rights as it would be backwards otherwise.
Then it draws the quad for the pause menu to where the projection will be displayed on. Then it restores everything to a 3d state allowing everything else to still be rendered 3d by the code.

Next the functionality of the pause menu needs to be implemented

```
340     # Main loop
341     while True:
342         for event in pygame.event.get():
343             if event.type == pygame.QUIT:
344                 pygame.quit()
345                 quit()
346             elif event.type == pygame.KEYDOWN:
347                 if event.key == pygame.K_ESCAPE:
348                     paused = not paused  # Toggle pause state
349                 elif event.key == pygame.K_q:  # Check for Q key
350                     if paused == True:
351                         pygame.quit()  # Quit the program
352                         quit()  # Exit the program
353 
354         # If the game is paused, display the pause menu and skip game update
355         if paused:
356             display_pause_menu()
357             pygame.display.flip()  # Update the display with the pause overlay
358             continue  # Skip the rest of the game update loop
```

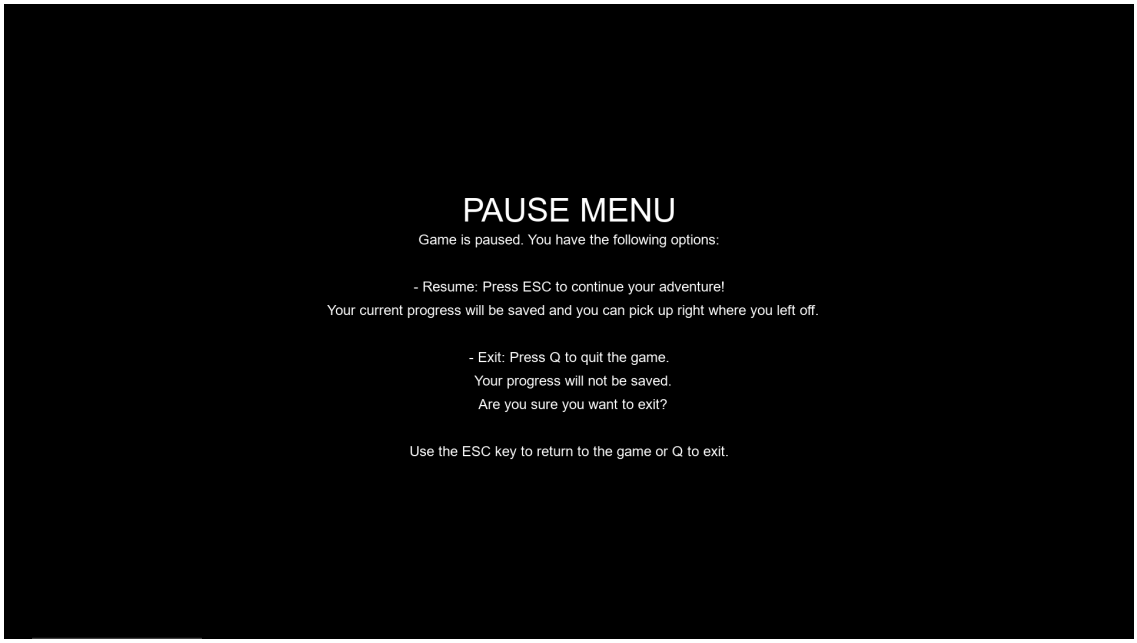
Line 346 checks if a key has been pressed and if that key that is pressed escapes, then it will toggle the pause state. I have used this as it follows a normal game conventions

Line 349 and below then check the same and see if the key Q was pressed, and if it was, close the window. Q was chosen as it is the start of the word quit and voted on by my stakeholders.

Lines 355-358 checks if the pause state is active, then if it is, it displays the pause menu and updates the window to display it properly. Then skips the rest of the game loop.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

The pause menu now generates like so:



Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Stage Four: The Timer

The 3d platformer will want a timer so there can be competition when people play to see if they can beat the fastest time for the level. The following code does this.

```
281 def draw_textured_quad(texture, width, height, position=(10, 10)):
282     glEnable(GL_TEXTURE_2D)
283     glBindTexture(GL_TEXTURE_2D, texture)
284     glColor3f(1, 1, 1)  # Use white color to keep the original texture color
285 
286     # Apply mirroring transformation by scaling negatively on the X-axis
287     glPushMatrix()
288     # Translate to the center of the text quad, mirror, then translate back
289     glTranslatef(position[0] + width / 2, position[1] + height / 2, 0)
290     glScalef(-1, 1, 1)  # Mirror horizontally by scaling X by -1
291     glRotatef(180, 0, 0, 1) # Rotate 180 degrees around the Z-axis
292     glTranslatef(-position[0] - width / 2, -position[1] - height / 2, 0)
293 
294     # Draw the textured quad
295     glBegin(GL_QUADS)
296     glTexCoord2f(0, 1)
297     glVertex2f(position[0], position[1] + height)
298     glTexCoord2f(1, 1)
299     glVertex2f(position[0] + width, position[1] + height)
300     glTexCoord2f(1, 0)
301     glVertex2f(position[0] + width, position[1])
302     glTexCoord2f(0, 0)
303     glVertex2f(position[0], position[1])
304     glEnd()
305 
306     glPopMatrix()
307     glDisable(GL_TEXTURE_2D)
```

Line 282-284 are set up to make the timer white as it is planned to be against a black background.

Lines 287-292 again sort out the weird rotation on the text and make it display correctly by rotating and mirroring it.

Linse 295-304 set up the quad that will be used to display the time in.

Now the timer needs to actually implemented into the code

```
334     # Timer variables
335     start_ticks = pygame.time.get_ticks()  # Timer start point
336     elapsed_time = 0  # Time elapsed in seconds
337     paused = False  # Pause state
```

Line 335 is the line that initialises the timer and actually starts it when the main loop activates.
Line 336 is where the elapsed time will be actually stored to be displayed in the timer area.

Line 337 is for when the timer is in a paused state when the pause menu is active.

```
360         # Update the elapsed time only when not paused
361         elapsed_time = (pygame.time.get_ticks() - start_ticks) / 1000  # Time in seconds
```

Line 361 takes the game ticks picked up by the timer and converts them into seconds by dividing by 100.

```
378         # Draw the timer (displaying the frozen time when paused)
379         timer_text = f"Time: {elapsed_time:.2f} s"
380         font = pygame.font.SysFont("Arial", 32)  # Use an appropriate font size
381         text_texture, text_width, text_height = render_text_with_background(
382             font, timer_text, text_color=(255, 255, 255), bg_color=(0, 0, 0, 150)  # Semi-transparent black background
383         )
```

Line 379 now writes the text for timer by taking elapsed_time and applying it to the screen.

Line 380 then applies the font and size.

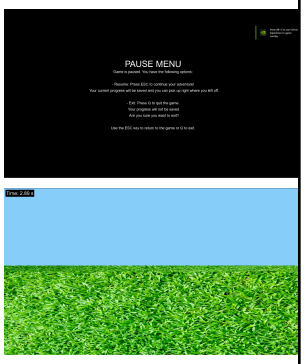
Line 381-382 now draws a black background for the timer to be displayed on, As the background colour is blue and no other colour in the game is black this makes sure the timer is visible no matter what.

This all together makes the timer generate in the top left corner of the screen like so:

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

TEST 3

test	code	expected	Result

Pause menu activation and deactivation	elif event.type == pygame.KEYDOWN:
	  if event.key == pygame.K_ESCAPE:
	  paused = not paused
		When escape key is pressed the pause menu should show and then when pressed again it should disappear	

Quit function	elif event.key == pygame.K_q: # Check for Q key if paused == True: pygame.quit() # Quit the program quit() # Exit the program	When the Q key is pressed the elif statement should activate and then a new if statement should check if the game is paused only then allowing the game to quit	Quit result: the game has been paused while q is pressed so the game is quitting
Timer paused	elapsed_time = (pygame.time.get_ticks() - start_ticks) / 1000	This gets the time for when the game is not paused and paused and subtracts off each other to get the right time	Failed: timer keeps updating when paused

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Stakeholder Feedback

I have spoken to my stakeholders and asked questions to get feedback on the first iteration of the game and where I can improve it.

1) How did the Movement feel?
    - The summary of the average response was that the movement felt clunky and wasn't smooth, juddering when moving
2) What were their thoughts on the pause menu?
    - They said that the pause menu was over complicated and had too many details and they wanted it to be simplified.
3) What about the timer function?
    - They said that the timer function works as intended but it is laggy and doesn't work when the pause menu is active, rendering the pause menu ineffective
4) What did you think about the ground generation?
    - They responded telling me how they thought the ground wasn't generated in a good way and was relevant to the player position and it moved when tested at different ground levels, When different player spawn levels were set it generated under the player still not at ground level which helped with collision. They wanted the ground to be one face instead of this and stuck to a fixed position.
5) What did you think about the camera?
    - They said while they liked the idea, the camera felt restrictive and they couldn't properly look around. They said they would prefer a third person view.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Iteration 2

Phase one: The Ground

I was having trouble generating the cubes into one solid platform that worked with the collision model I wanted to use. There were spaces between the cubes causing the player character to get stuck. So I decided to get rid of my old generation for cubes and the ground and rework them to fit my model better.

```
85  def draw_ground():
86      if ground_texture is None:
87          # Fallback color if texture isn't loaded
88          glDisable(GL_TEXTURE_2D)
89          glColor3f(0.0, 0.8, 0.0)  # Green ground
90      else:
91          glEnable(GL_TEXTURE_2D)
92          glColor3f(1.0, 1.0, 1.0)  # White color to not tint texture
93          glBindTexture(GL_TEXTURE_2D, ground_texture)
94
95      glBegin(GL_QUADS)
96      # Ground plane with texture coordinates
97      glTexCoord2f(0.0, 0.0)
98      glVertex3f(-5, 0, 5)  # Near left
99      glTexCoord2f(5.0, 0.0)
100     glVertex3f(5, 0, 5)   # Near right
101     glTexCoord2f(5.0, 5.0)
102     glVertex3f(5, 0, -5)  # Far right
103     glTexCoord2f(0.0, 5.0)
104     glVertex3f(-5, 0, -5) # Far left
105     glEnd()
106
107     glDisable(GL_TEXTURE_2D)
```

Lines 86-93 all work together to either apply a texture to the ground plane that will be generated or instead make the plane green. By checking if the texture can be loaded and if it can, load it.

Lines 95-105 are all the texture coords and vertex coords for the ground plane to generate for the texture to be applied to.

This function now instead of generating one cube at a time and then merging them together to try and create a seamless ground with no gaps, instead now creates a one face ground plane for the grass texture to load on. With its size editable by changing the numbers in glVertex3f. This gives the player a flat ground to walk across that is seamless and has no gaps in between. This was liked by the stakeholders over the old version as it removes many of the issues with the old ground.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Phase two: The Platforms

My stakeholders had another issue with my first version of the game, there was nowhere to actually jump to when moving. The player could jump, but they couldn't jump between platforms, which is the main aim of a 3d platformer. So this fixes that.

```
109 def draw_platform():
110     glPushMatrix()
111     glTranslatef(2, 1, -2)  # Position of the platform
112     glScalef(2, 0.2, 2)    # Size of the platform
113     glBegin(GL_QUADS)
114     glColor3f(0.5, 0.5, 0.5)  # Gray color
115 
116     # Top face
117     glVertex3f(-0.5, 0.5, -0.5)
118     glVertex3f(0.5, 0.5, -0.5)
119     glVertex3f(0.5, 0.5, 0.5)
120     glVertex3f(-0.5, 0.5, 0.5)
121 
122     # Bottom face
123     glVertex3f(-0.5, -0.5, -0.5)
124     glVertex3f(0.5, -0.5, -0.5)
125     glVertex3f(0.5, -0.5, 0.5)
126     glVertex3f(-0.5, -0.5, 0.5)
127 
128     # Front face
129     glVertex3f(-0.5, -0.5, 0.5)
130     glVertex3f(0.5, -0.5, 0.5)
131     glVertex3f(0.5, 0.5, 0.5)
132     glVertex3f(-0.5, 0.5, 0.5)
133 
134     # Back face
135     glVertex3f(-0.5, -0.5, -0.5)
136     glVertex3f(0.5, -0.5, -0.5)
137     glVertex3f(0.5, 0.5, -0.5)
138     glVertex3f(-0.5, 0.5, -0.5)
139 
140     # Right face
141     glVertex3f(0.5, -0.5, -0.5)
142     glVertex3f(0.5, 0.5, -0.5)
143     glVertex3f(0.5, 0.5, 0.5)
144     glVertex3f(0.5, -0.5, 0.5)
145 
146     # Left face
147     glVertex3f(-0.5, -0.5, -0.5)
148     glVertex3f(-0.5, 0.5, -0.5)
149     glVertex3f(-0.5, 0.5, 0.5)
150     glVertex3f(-0.5, -0.5, 0.5)
151 
152     glEnd()
153     glPopMatrix()
```

Lines 110 - 114 sets up where the first platform will be generated, the scale so how big that platform will be and the colour of the platform to be grey.

Then each comment sets out how each face for the platform will be generated and the vertex positions of it all, so the platform can be the right width and height.

This now gives the player a goal to reach and jump to, now the mechanic is there for the player to jump between the platforms and now gives me the platform ~~for~~ in iteration three to develop and expand the platform system.

Phase Three: My player

Again there were problems when testing the game over the character with how laggy and buggy the movement was when traversing the original version so I had to implement a few fixes to make the game load smoother. But also changed how the player is made. I originally used Object Oriented Coding to create the player and the camera, but I have decided to change this now as it is more simple for the player to be generated without it.

```
155 def handle_movement():
156     global player_pos, player_velocity, is_jumping, is_paused
157     keys = pygame.key.get_pressed()
158 
159     if not is_paused:
160         # Movement Controls
161         if keys[K_w]:
162             player_pos[2] -= player_speed
163         if keys[K_s]:
164             player_pos[2] += player_speed
165         if keys[K_a]:
166             player_pos[0] -= player_speed
167         if keys[K_d]:
168             player_pos[0] += player_speed
169 
170         # Jumping
171         if not is_jumping and keys[K_SPACE]:
172             player_velocity[1] = jump_power
173             is_jumping = True
174 
175         # Apply gravity
176         player_velocity[1] += gravity
177         player_pos[1] += player_velocity[1]
178 
```

This mostly takes the ideas from the original movement code and implements in a new way by making sure the keys work and using new variables and Constants made at the start of the code to calculate the speed and jump power.

Gravity works in the same way by using a constant to have a solid gravity power and then calculates what is the player velocity and the height of the player pos by changing the float.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Now I need to implement the new collision model which is needed so the player can actually change levels and drop off the map instead of the collision being on y = 0 across the board.

```
        # Check for collisions with ground
        if player_pos[1] <= 0.5:
            player_pos[1] = 0.5
            is_jumping = False
            player_velocity[1] = 0

        # Check for collisions with platform
        platform_pos = [2, 1, -2]  # Position of the platform
        platform_size = [2, 0.2, 2]  # Size of the platform
        collision_offset = 0.4  # Collision offset value
        if (
            platform_pos[0] - platform_size[0] / 2 - collision_offset <= player_pos[0] <= platform_pos[0] + platform_size[0] / 2 + collision_offset and
            platform_pos[1] - collision_offset <= player_pos[1] <= platform_pos[1] + platform_size[1] + collision_offset and
            platform_pos[2] - platform_size[2] / 2 - collision_offset <= player_pos[2] <= platform_pos[2] + platform_size[2] / 2 + collision_offset
        ):
            player_pos[1] = platform_pos[1] + platform_size[1] + collision_offset
            is_jumping = False
            player_velocity[1] = 0

        # Check if player falls off the map (for simplicity)
        if abs(player_pos[0]) > 5 or abs(player_pos[2]) > 5:
            player_pos = [0, 2, 0]  # Reset position after falling
        glLoadIdentity()  # Reset the model view matrix
        glTranslatef(-player_pos[0], -player_pos[1], -player_pos[2] - 5)
```

When checking for the ground collision, it is set the same as before, where the ground is at a set height equal to where the plane for my ground is.

But as the Platforms will be on different levels it is needed for the collision to be different also. To do this I set a collision off-set value and help use that to then calculate the platform positions relative to the player to see if the player will collide with it.

This then stops the player jumping and sets the value to 0 and turns is_jumping to false so the player can jump again.

I also don't want the player going out of bounds of the map so I created a system if the player leaves the level area, it will reset the player towards the centre for them to start again.

Now I will generate the player in this new model still making it a cube

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

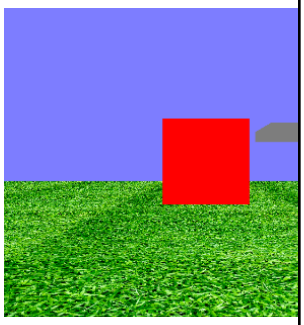
```
def draw_player():
    glPushMatrix()
    glTranslatef(player_pos[0], player_pos[1], player_pos[2])
    glBegin(GL_QUADS)
    glColor3f(1, 0, 0)  # Red color

    # Front face
    glVertex3f(-0.5, -0.5, -0.5)
    glVertex3f(0.5, -0.5, -0.5)
    glVertex3f(0.5, 0.5, -0.5)
    glVertex3f(-0.5, 0.5, -0.5)

    # Back face
    glVertex3f(-0.5, -0.5, 0.5)
    glVertex3f(0.5, -0.5, 0.5)
    glVertex3f(0.5, 0.5, 0.5)
    glVertex3f(-0.5, 0.5, 0.5)

    # Top face
    glVertex3f(-0.5, 0.5, -0.5)
    glVertex3f(0.5, 0.5, -0.5)
    glVertex3f(0.5, 0.5, 0.5)
    glVertex3f(-0.5, 0.5, 0.5)

    # Bottom face
    glVertex3f(-0.5, -0.5, -0.5)
    glVertex3f(0.5, -0.5, -0.5)
    glVertex3f(0.5, -0.5, 0.5)
    glVertex3f(-0.5, -0.5, 0.5)

    # Right face
    glVertex3f(0.5, -0.5, -0.5)
    glVertex3f(0.5, 0.5, -0.5)
    glVertex3f(0.5, 0.5, 0.5)
    glVertex3f(0.5, -0.5, 0.5)

    # Left face
    glVertex3f(-0.5, -0.5, -0.5)
    glVertex3f(-0.5, 0.5, -0.5)
    glVertex3f(-0.5, 0.5, 0.5)
    glVertex3f(-0.5, -0.5, 0.5)
```

This, like the platform, generates all the faces of the cube separately and then combines them into a cube, then the positions of the corners are then used in the player pos variable to give an accurate collision box for the player.

TEST 4

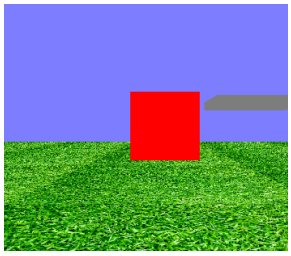
test	code	expected	Result
Load the texture	texture = glGenTextures(1) img = pygame.image.load (filename)	The program should output a success message and the texture should load onto the ground	Successfully loaded texture: 1

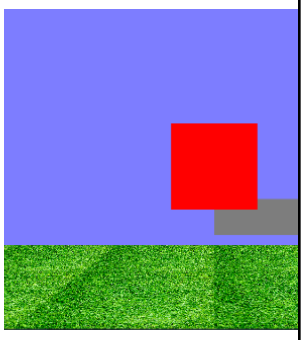
Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

test	code	expected	Result
			

Check ground generation	glBegin(GL_QUADS)
	    # Ground plane with texture coordinates
	
	glTexCoord2f(0.0, 0.0)
	    glVertex3f(-5, 0, 5) # Near left
	
	glTexCoord2f(5.0, 0.0)
	    glVertex3f(5, 0, 5) # Near right
	
	glTexCoord2f(5.0, 5.0)
	    glVertex3f(5, 0, -5) # Far right
	
	glTexCoord2f(0.0, 5.0)
	    glVertex3f(-5, 0, -5) # Far left
	    glEnd()
		This will draw a ground that will 9 x9 in size to hold the player with the ground generating on y = 0	

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

test	code	expected	Result
Platform generation	pg.22	A platform at the specified height will appear and will be the colour grey	
Player generation	Pg. 23	A red cube will generate as a player that can be moved around with the movement keys	
Collisions with ground	# Check for collisions with ground if player_pos[1] <= 0.5: player_pos[1] = 0.5 is_jumping = False player_velocity[1] = 0	The player when spawning in will fall to the ground level at 0.5 to give the illusion of standing on the ground without the player box clipping into the ground.	

Check collisions with Platform	platform_pos = [2, 1, -2] # Position of the platform
	platform_size = [2, 0.2, 2] # Size of the platform
	collision_offset = 0.4 # Collision offset value
	if (
	
	platform_pos[0] - platform_size[0] / 2
		The player will be able to stand on the platform with no problem and when they try to step off the player should fall off and collide with the floor	

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

test	code	expected	Result
	- collision_offset <= player_pos[0] <= platform_pos[0] + platform_size[0] / 2 + collision_offset and platform_pos[1] - collision_offset <= player_pos[1] <= platform_pos[1] + platform_size[1] + collision_offset and platform_pos[2] - platform_size[2] / 2 - collision_offset <= player_pos[2] <= platform_pos[2] + platform_size[2] / 2 + collision_offset): player_pos[1] = platform_pos[1] + platform_size[1] + collision_offset is_jumping = False player_velocity[1] = 0		

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

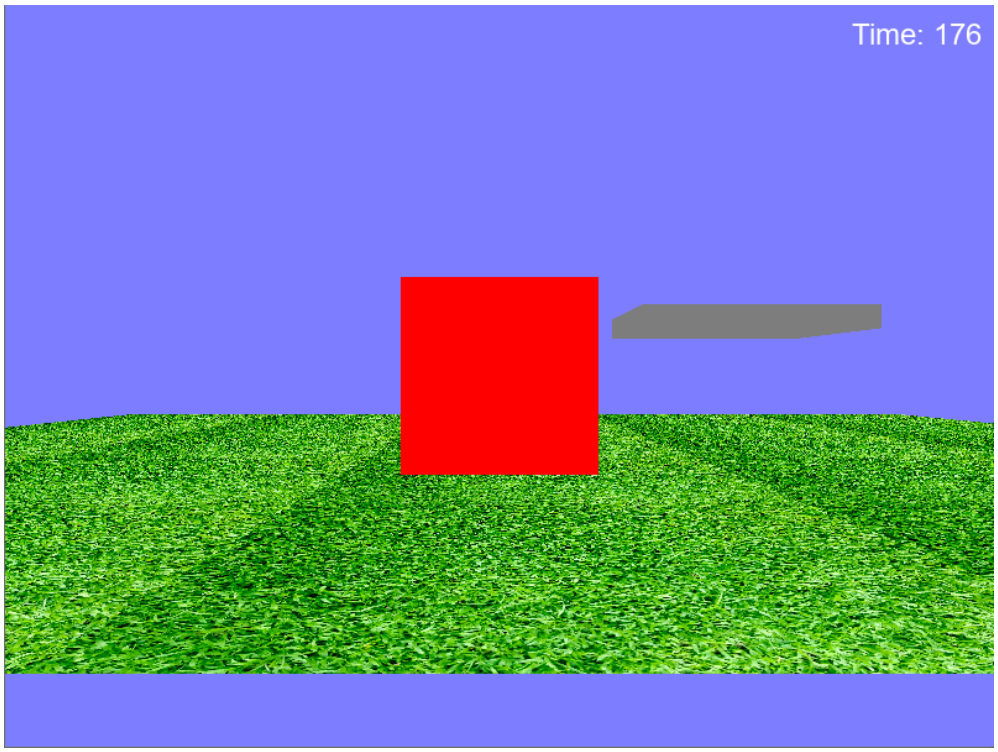
Phase Four: The Timer

The Timer had some feedback with the timer looking too ‘ugly’ in the eyes of my stakeholders, so they asked me to remove the black background that was surrounding it and shorten the amount of time the timer shows.
They also wanted me to move it to the top left corner

```
250 def draw_timer():
251     global elapsed_time
252     if not is_paused:
253         current_time = time.time()
254         elapsed_time = stored_time + (current_time - start_time)
255 
256     # Save the current matrices and attributes
257     glMatrixMode(GL_PROJECTION)
258     glPushMatrix()
259     glLoadIdentity()
260     glOrtho(0, WINDOW_WIDTH, WINDOW_HEIGHT, 0, -1, 1)
261     glMatrixMode(GL_MODELVIEW)
262     glPushMatrix()
263     glLoadIdentity()
264 
265     # Disable depth testing temporarily
266     glDisable(GL_DEPTH_TEST)
267 
268     # Render the timer text
269     font = pygame.font.SysFont('Arial', 24)
270     timer_text = font.render(f"Time: {int(elapsed_time)}", True, (255, 255, 255))
271     timer_surface = pygame.Surface((timer_text.get_width(), timer_text.get_height()), pygame.SRCALPHA)
272     timer_surface.blit(timer_text, (0, 0))
273 
274     # Convert the surface to a string
275     timer_data = pygame.image.tostring(timer_surface, "RGBA", True)
276 
277     # Generate and bind a new texture
278     timer_texture = glGenTextures(1)
279     glBindTexture(GL_TEXTURE_2D, timer_texture)
280 
281     # Set texture parameters
282     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
283     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
284 
285     # Load the texture
286     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, timer_surface.get_width(), timer_surface.get_height(),
287                  0, GL_RGBA, GL_UNSIGNED_BYTE, timer_data)
288 
289     # Draw the texture as a quad in 2D space
290     glEnable(GL_TEXTURE_2D)
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
291     glEnable(GL_BLEND)
292     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
293
294     x = WINDOW_WIDTH - timer_surface.get_width() - 10
295     y = 10
296
297     glColor4f(1.0, 1.0, 1.0, 1.0)  # Set color to white with full opacity
298
299     glBegin(GL_QUADS)
300     glTexCoord2f(0, 1); glVertex2f(x, y)
301     glTexCoord2f(1, 1); glVertex2f(x + timer_surface.get_width(), y)
302     glTexCoord2f(1, 0); glVertex2f(x + timer_surface.get_width(), y + timer_surface.get_height())
303     glTexCoord2f(0, 0); glVertex2f(x, y + timer_surface.get_height())
304     glEnd()
305
306     # Clean up
307     glDeleteTextures([timer_texture])
308     glDisable(GL_BLEND)
309     glDisable(GL_TEXTURE_2D)
310     glEnable(GL_DEPTH_TEST)
311
312     # Restore the matrices
313     glMatrixMode(GL_PROJECTION)
314     glPopMatrix()
315     glMatrixMode(GL_MODELVIEW)
316     glPopMatrix()
```



This is what the output looks like with these 4 phases completed.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

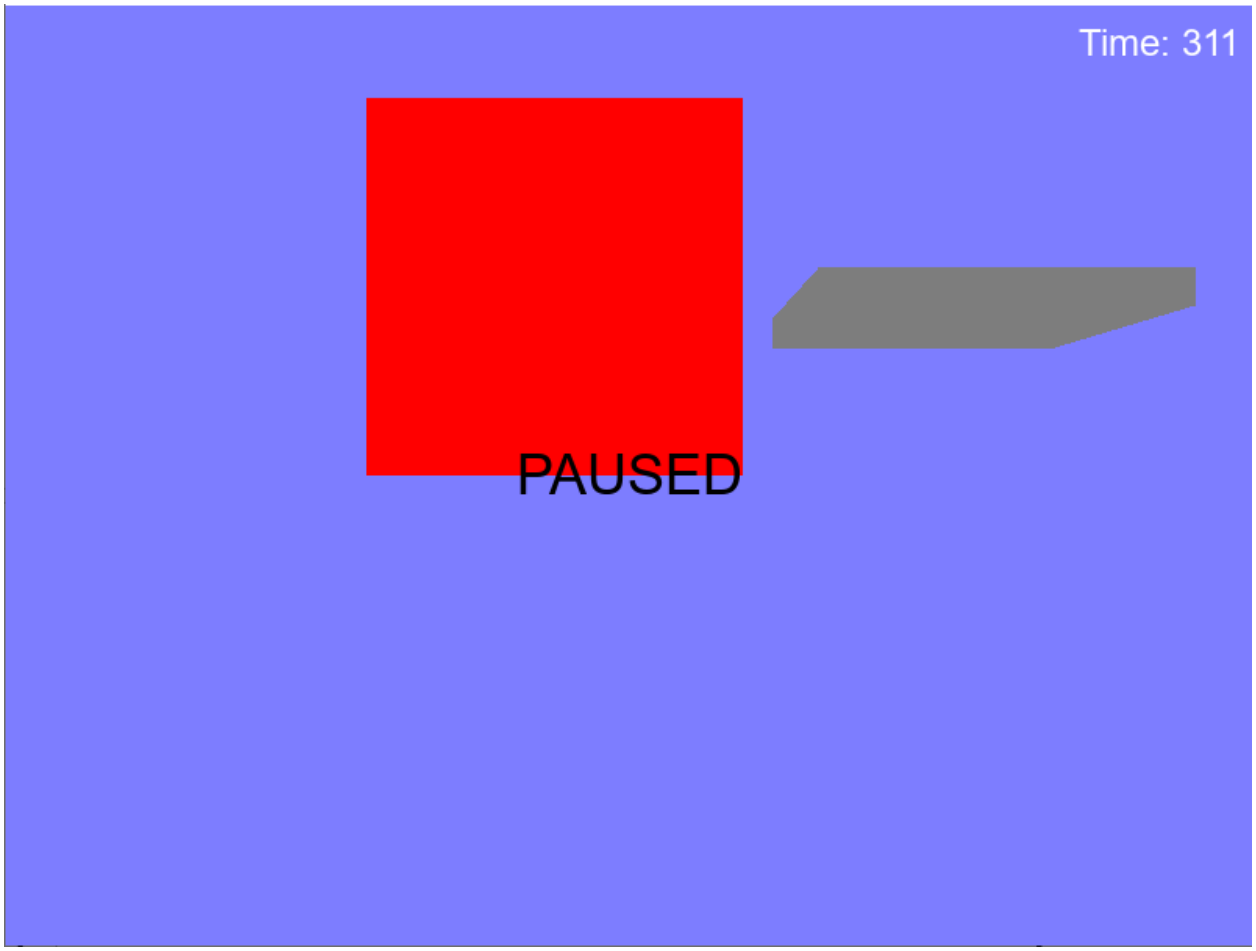
Phase Five: The Pause Menu

There was also some feedback over the pause menu again saying it didn't look right. Instead the stakeholders wanted to now a see through background with only text saying paused on the screen. This code achieves this.

```
318 def pause_menu():
319     # Save the current matrices and attributes
320     glMatrixMode(GL_PROJECTION)
321     glPushMatrix()
322     glLoadIdentity()
323     glOrtho(0, WINDOW_WIDTH, WINDOW_HEIGHT, 0, -1, 1)
324     glMatrixMode(GL_MODELVIEW)
325     glPushMatrix()
326     glLoadIdentity()
327 
328     # Disable depth testing temporarily
329     glDisable(GL_DEPTH_TEST)
330 
331     # Render the pause text
332     font = pygame.font.SysFont('Arial', 36)
333     pause_text = font.render("PAUSED", True, (0, 0, 0))
334     pause_surface = pygame.Surface((pause_text.get_width(), pause_text.get_height()), pygame.SRCALPHA)
335     pause_surface.blit(pause_text, (0, 0))
336 
337     # Convert the surface to a string
338     pause_data = pygame.image.tostring(pause_surface, "RGBA", True)
339 
340     # Generate and bind a new texture
341     pause_texture = glGenTextures(1)
342     glBindTexture(GL_TEXTURE_2D, pause_texture)
343 
344     # Set texture parameters
345     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
346     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
347 
348     # Load the texture
349     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, pause_surface.get_width(), pause_surface.get_height(),
350                  0, GL_RGBA, GL_UNSIGNED_BYTE, pause_data)
351 
352     # Draw the texture as a quad in 2D space
353     glEnable(GL_TEXTURE_2D)
354     glEnable(GL_BLEND)
355     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
356 
357     x = (WINDOW_WIDTH - pause_surface.get_width()) // 2
358     y = (WINDOW_HEIGHT - pause_surface.get_height()) // 2
```

```
359 
360     glColor4f(1.0, 1.0, 1.0, 1.0)  # Set color to white with full opacity
361 
362     glBegin(GL_QUADS)
363     glTexCoord2f(0, 1); glVertex2f(x, y)
364     glTexCoord2f(1, 1); glVertex2f(x + pause_surface.get_width(), y)
365     glTexCoord2f(1, 0); glVertex2f(x + pause_surface.get_width(), y + pause_surface.get_height())
366     glTexCoord2f(0, 0); glVertex2f(x, y + pause_surface.get_height())
367     glEnd()
368 
369     # Clean up
370     glDeleteTextures([pause_texture])
371     glDisable(GL_BLEND)
372     glDisable(GL_TEXTURE_2D)
373     glEnable(GL_DEPTH_TEST)
374 
375     # Restore the matrices
376     glMatrixMode(GL_PROJECTION)
377     glPopMatrix()
378     glMatrixMode(GL_MODELVIEW)
379     glPopMatrix()
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework



Phase Six: The Main Game Loop

The old game loop in Iteration one, was very long and cluttered, leading to the game having performance issues. So I decided to move a lot of the functions to the a new function called Handle Events.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
def handle_events():
    global is_paused, start_time, stored_time, elapsed_time
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            quit()
        if event.type == pygame.KEYDOWN:
            if event.key == K_ESCAPE:
                is_paused = not is_paused
                if is_paused:
                    # Store the current elapsed time when pausing
                    stored_time = elapsed_time
                else:
                    # When unpausing, adjust start_time so elapsed_time continues from stored_time
                    start_time = time.time() - stored_time

            if event.key == K_q and is_paused:
                pygame.quit()
                quit()
```

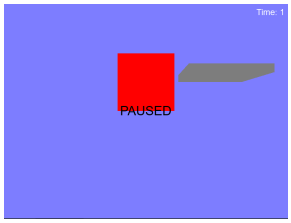
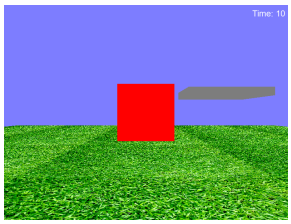
This handles the timer and pause features. When the correct keys are pressed the actions to pause and unpause or quit happen.

There is also the added feature of when pausing, the timer stops, and is stored until the game is unpaused. This fixes an issue with the first timer, where no matter what happened the timer was always running. Now the timer stops when the game is paused, so works as intended.

```
def main():
    global ground_texture, is_paused
    ground_texture = load_texture('grass.jpg')
    clock = pygame.time.Clock()

    try:
        while True:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    raise SystemExit
                elif event.type == pygame.VIDEORESIZE:
                    resize_window(event.w, event.h)
                elif event.type == pygame.KEYDOWN:
                    if event.key == K_ESCAPE:
                        is_paused = not is_paused
                    if event.key == K_q and is_paused:
                        raise SystemExit

            # Clear the screen
            glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

            if not is_paused:
                glLoadIdentity()  # Reset the modelview matrix
                glTranslatef(0.0, 0.0, -5)  # Move back into the scene
                handle_movement()
                draw_ground()
                draw_platform()
                draw_player()
                draw_timer()
            else:
                glLoadIdentity()  # Reset the modelview matrix
                glTranslatef(0.0, 0.0, -5)  # Move back into the scene
                draw_ground()
                draw_platform()
                draw_player()
                draw_timer()
                pause_menu()

            pygame.display.flip()
            clock.tick(60)
```

Now the main game loop is a lot more simplified and easy to understand with little clutter. The game now uses functions separated out from all of the game code. While using an if statement to check if the game is paused and runs the correct functions.

TEST 5

test	code	expected	Result
Pause menu load	if event.key == K_ESCAPE: is_paused = not is_paused	Loads the pause screen	
Pause Menu deactive	if event.key == K_ESCAPE: is_paused = not is_paused	Closes the pause screen	

Timer stops when paused	if is_paused:
	# Store the current elapsed time when pausing
	
	stored_time = elapsed_time
	else:
	# When unpausing, adjust start_time so elapsed_time continues from stored_time
	
	start_time = time.time() - stored_time
		If the game is paused it should grab the time when paused in ticks and save that waiting for the game to unpause	Timer carries on going and doesn't stop. FAIL

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Stakeholder Feedback

My stakeholders are a lot happier with this version of the game with the feedback being a lot more positive than the first iteration.

1) How was the ground generation?
    - The response was positive now that the ground was only one face and had a better collision model as it could now not let the camera through it and generated under the player, but not the biggest fan of as soon as the player leaves the area no matter the height the player is teleported back to the centre. And its sudden as soon as one part of the player falls off the area. Not after falling under the play area specified as ground level.
2) What did you think of the platform?
    - They liked the platform generation and how it looked compared to the rest and how it stood out., But would want a colour change and also more variations implemented as suggested by the design of this document
3) What did you think about the camera?
    - They are happy with the camera and the movement now and with it being able to collide with the objects around the level but want to be able to actually move the camera around.
4) What did you think about the timer?
    - The timer is now smoother and now the timer works under pause menu conditions.
5) What did you think about the movement?
    - The movement feels smoother now and they felt that the player reacted well to the environment with the movement not jolting around anymore.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Iteration 3

Phase One: The walls

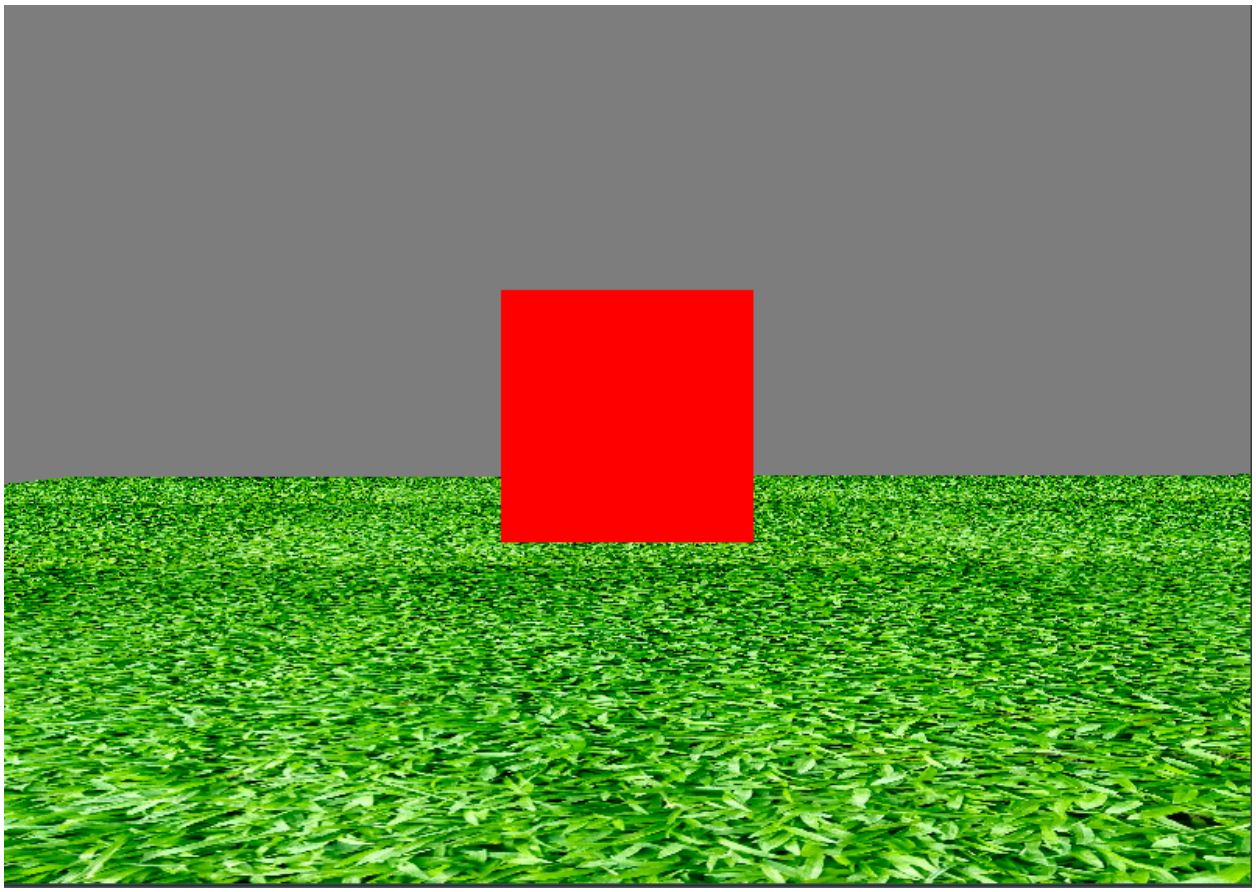
My Stakeholders had complaints over the reset system, which when leaving the area was being too sensitive and activating too soon. So, as a fix, I decided to add a wall to surround the ground to allow a better way to stop the players from falling off the ground and going out of bounds.

```
218 def draw_walls():
219     wall_height = 25.0  # Height of the walls
220     ground_size = 7.0  # Size of the ground (matches your GROUND_SIZE)
221 
222     glColor3f(0.5, 0.5, 0.5)  # Gray color for walls
223     glBegin(GL_QUADS)
224 
225     # North wall
226     glVertex3f(-ground_size, 0, -ground_size)
227     glVertex3f(ground_size, 0, -ground_size)
228     glVertex3f(ground_size, wall_height, -ground_size)
229     glVertex3f(-ground_size, wall_height, -ground_size)
230 
231     # South wall
232     glVertex3f(-ground_size, 0, ground_size)
233     glVertex3f(ground_size, 0, ground_size)
234     glVertex3f(ground_size, wall_height, ground_size)
235     glVertex3f(-ground_size, wall_height, ground_size)
236 
237     # East wall
238     glVertex3f(ground_size, 0, -ground_size)
239     glVertex3f(ground_size, 0, ground_size)
240     glVertex3f(ground_size, wall_height, ground_size)
241     glVertex3f(ground_size, wall_height, -ground_size)
242 
243     # West wall
244     glVertex3f(-ground_size, 0, -ground_size)
245     glVertex3f(-ground_size, 0, ground_size)
246     glVertex3f(-ground_size, wall_height, ground_size)
247     glVertex3f(-ground_size, wall_height, -ground_size)
248 
249     glEnd()
```

By using two new variables for wall height and ground size, I can calculate how to generate the walls in the right way. The walls go around the size of the ground and to the specified height. Then the collisions are implemented the same as how the platforms are done.

This helps to solve the problem of the sensitivity of the reset from falling off, so now they don’t have to worry as the walls block the player falling

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework



Phase Two: The Camera

With how the camera was generated before, it was locked to one angle looking at the face of the cube meaning the player can not look around the scene.
This was a complaint from my stakeholders, so I decided to implement a 3rd person camera, that will follow the back of the cube and can rotate the cube as the player moves the mouse. But it doesn't flip the cube upside down looking up or down so there has to be a limit on the y-axis.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
251 def handle_camera():
252     global camera_angle_x, camera_angle_y, player_rotation
253 
254     # Get mouse movement
255     mouse_rel = pygame.mouse.get_rel()
256     camera_angle_x -= mouse_rel[0] * camera_sensitivity
257     camera_angle_y -= mouse_rel[1] * camera_sensitivity
258 
259     # Keep player rotation synced with camera
260     player_rotation = camera_angle_x
261 
262     # Limit vertical camera rotation
263     camera_angle_y = max(min(camera_angle_y, math.pi/3), -math.pi/3)
264 
265     # Calculate camera position to orbit around the cube
266     camera_distance = 5
267 
268     # Calculate camera direction vector (from player to camera)
269     dir_x = math.cos(camera_angle_y) * math.sin(camera_angle_x)
270     dir_y = math.sin(camera_angle_y)
271     dir_z = math.cos(camera_angle_y) * math.cos(camera_angle_x)
272 
273     # Calculate initial camera position
274     camera_x = player_pos[0] + camera_distance * dir_x
275     camera_y = player_pos[1] + camera_distance * dir_y
276     camera_z = player_pos[2] + camera_distance * dir_z
277 
278     # Prevent camera from going below ground
279     if camera_y < 0.5:
280         camera_y = 0.5
281 
282     # Check collision with platforms
283     for platform in PLATFORMS:
284         # Calculate platform boundaries
285         platform_min_x = platform['pos'][0] - platform['size'][0]/2
286         platform_max_x = platform['pos'][0] + platform['size'][0]/2
287         platform_min_z = platform['pos'][2] - platform['size'][2]/2
288         platform_max_z = platform['pos'][2] + platform['size'][2]/2
289         platform_top = platform['pos'][1] + platform['size'][1]
```

```
290 
291 
292 
293     # Add camera wall collision detection
294     wall_buffer = 0.5
295     if camera_x > GROUND_SIZE - wall_buffer:
296         camera_x = GROUND_SIZE - wall_buffer
297     if camera_x < -GROUND_SIZE + wall_buffer:
298         camera_x = -GROUND_SIZE + wall_buffer
299     if camera_z > GROUND_SIZE - wall_buffer:
300         camera_z = GROUND_SIZE - wall_buffer
301     if camera_z < -GROUND_SIZE + wall_buffer:
302         camera_z = -GROUND_SIZE + wall_buffer
303 
304     # Look at the cube's position
305     glLoadIdentity()
306     gluLookAt(
307         camera_x, camera_y, camera_z,
308         player_pos[0], player_pos[1], player_pos[2],
309         0, 1, 0
310     )
```

This function starts at line 255 it gets the movement of the mouse by using the pygame.mouse.get_rel feature then takes the camera angle of x and y and takes the mouse rel and multiplying it by the constant value setup as 0.03 then taking this from camera angles.

A new feature is set up in the player to get the player rotation, then this is synced to the mouse movement. Now the player can rotate the cube with the mouse

Then On line 263 there is a limiter on the y-axis to make sure the camera doesn't flip on itself and can get behind the cube or rotate it in a weird way.

The variable for camera distance is set to 5, as this was found to be the optimum distance from the player for the camera to be.

Then some maths using sin and cos is used to figure out the angle on the camera, so it can be updated with the new camera angle as the mouse moves thanks to the earlier part where the mouse rel is.

It also then gets the initial camera angle, so by having the mouse position set as zero. This makes it look as a straight angle at the back facing of the cube.

I don't want the camera going through a ground, wall or platforms, so I needed to put collisions on the camera, so it can interact with them, which is done by calculating the height and width of the platforms, walls and ground.

Then if the camera tries to go inside the collision boundaries for them it will be blocked and it won't be able to pass into it, thanks to the collision offset putting the camera a bit away from the collision object.

Phase Three: The Platforms(again)

In my design section, I said I wanted to add different items like traps, boosts, checkpoints and a finish platform.

The following code shows how it can be implemented and how I can generate more than one platform.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
153 def draw_platforms():
154     current_time = time.time()
155     for platform in PLATFORMS:
156         platform_pos = tuple(platform['pos'])  # Convert list to tuple
157         # Skip drawing if it's a triggered trap that has expired
158         if platform.get('trap') and platform_pos in trap_timers:
159             if current_time - trap_timers[platform_pos] > TRAP_DURATION:
160                 continue
161 
162         glPushMatrix()
163         glTranslatef(*platform['pos'])
164         glScalef(*platform['size'])
165         glBegin(GL_QUADS)
166 
167         # Set color based on platform type
168         if platform.get('trap'):
169             glColor3f(0.8, 0.2, 0.2)  # Red for trap platforms
170         elif platform.get('boost'):
171             glColor3f(0.2, 0.8, 0.2)  # Green for boost platforms
172         elif platform.get('checkpoint'):
173             glColor3f(*CHECKPOINT_COLOR)  # Cyan for checkpoint platforms
174         elif platform.get('finish'):
175             glColor3f(*FINISH_COLOR)  # Gold for finish platform
176         else:
177             glColor3f(2.0, 0.5, 1.0)  # Gray for regular platforms
```

Line 155 - 159 is for the trap platforms, which uses a trap timer constant to start the timer for the platform to disappear. It does this by changing the pos of the platform when it is finished.

The rest is for making the colours of the platforms and how it looks. I have used red for trap platforms, Green for my boost platforms, cyan for the checkpoints and gold for the finished platforms.

The Boost platforms use a constant called BOOST_POWER which overrides the jump power. Then when the player jumps on the platform, they jump with that new jump power once then it goes back to the original jump power.

The Check Points work a bit differently, I decided to implement a reset limit allowing the player to click reset 3 times to return to last checkpoint or the start point if no checkpoint has been reached.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
453 def reset_to_checkpoint():
454     global player_pos, trap_timers, current_checkpoint, SCORE_SAVED, checkpoint_resets_remaining
455 
456     if checkpoint_resets_remaining <= 0:
457         return  # Don't allow reset if no resets remaining
458 
459     # Reset player position to last checkpoint or start position
460     if current_checkpoint:
461         player_pos = current_checkpoint.copy()
462     else:
463         player_pos = PLAYER_START_POS.copy()
464 
465     # Clear all trap timers to reset traps
466     trap_timers.clear()
467 
468     # Reset the score saved flag
469     SCORE_SAVED = False
470 
471     # Decrease remaining resets
472     checkpoint_resets_remaining -= 1
```

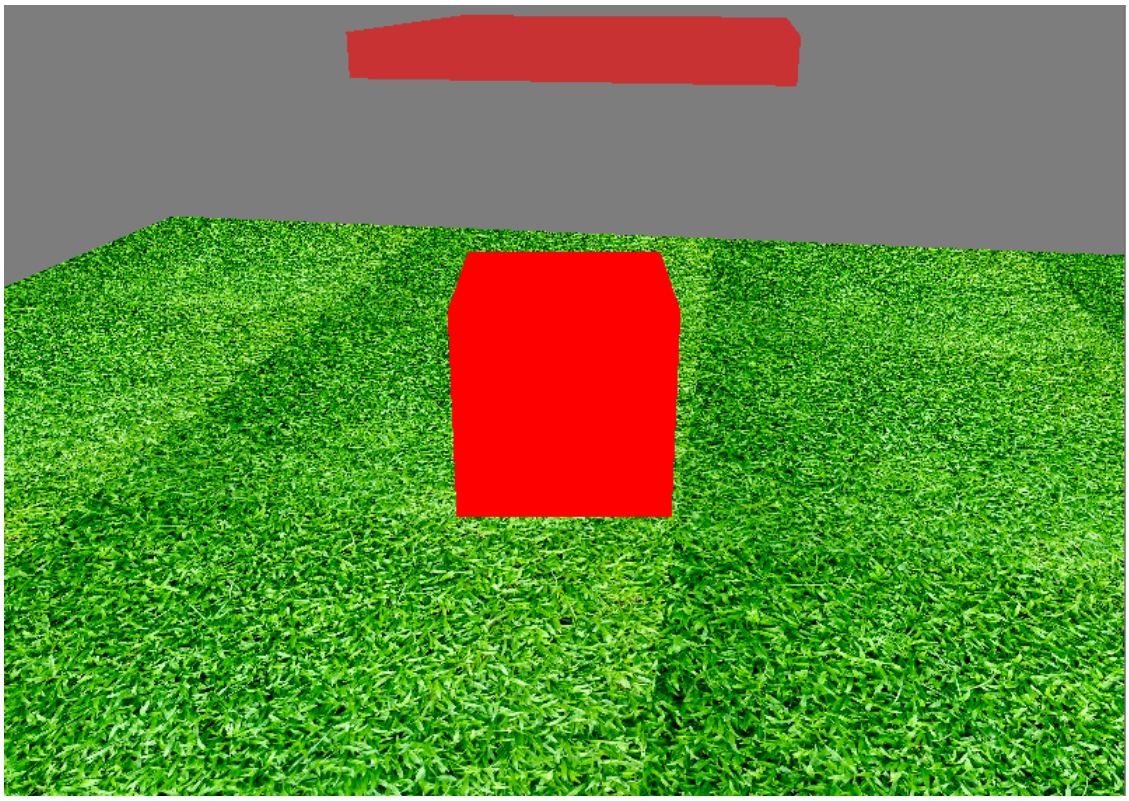
This saves the current position of the centre of the checkpoint. If there is still a reset left that can be done by the player, it overrides the player pos and sends them back to the checkpoint or start position.

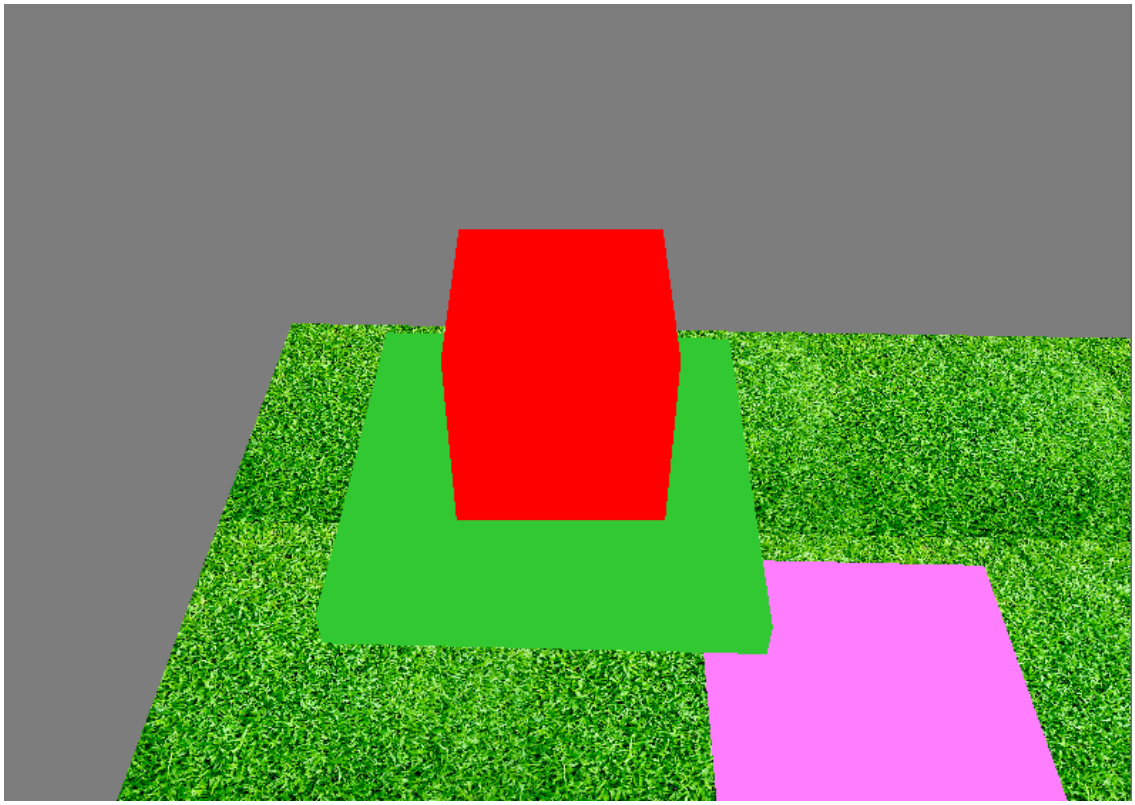
It also resets the traps as they would need to reset as they disappeared after a fail, while also lowering the checkpoint_resets_remaining number by one.

```
474 def draw_resets_remaining():
475     font = pygame.font.SysFont('Arial', 24)
476     resets_text = font.render(f"Resets: {checkpoint_resets_remaining}", True, (255, 255, 255))
477     resets_surface = pygame.Surface((resets_text.get_width(), resets_text.get_height()), pygame.SRCALPHA)
478     resets_surface.blit(resets_text, (0, 0))
479 
480     x = 10  # Position on the left side
481     y = 10  # Position at the top
482 
483     draw_2d_text([resets_surface], [(x, y)], [(resets_surface.get_width(), resets_surface.get_height())])
```

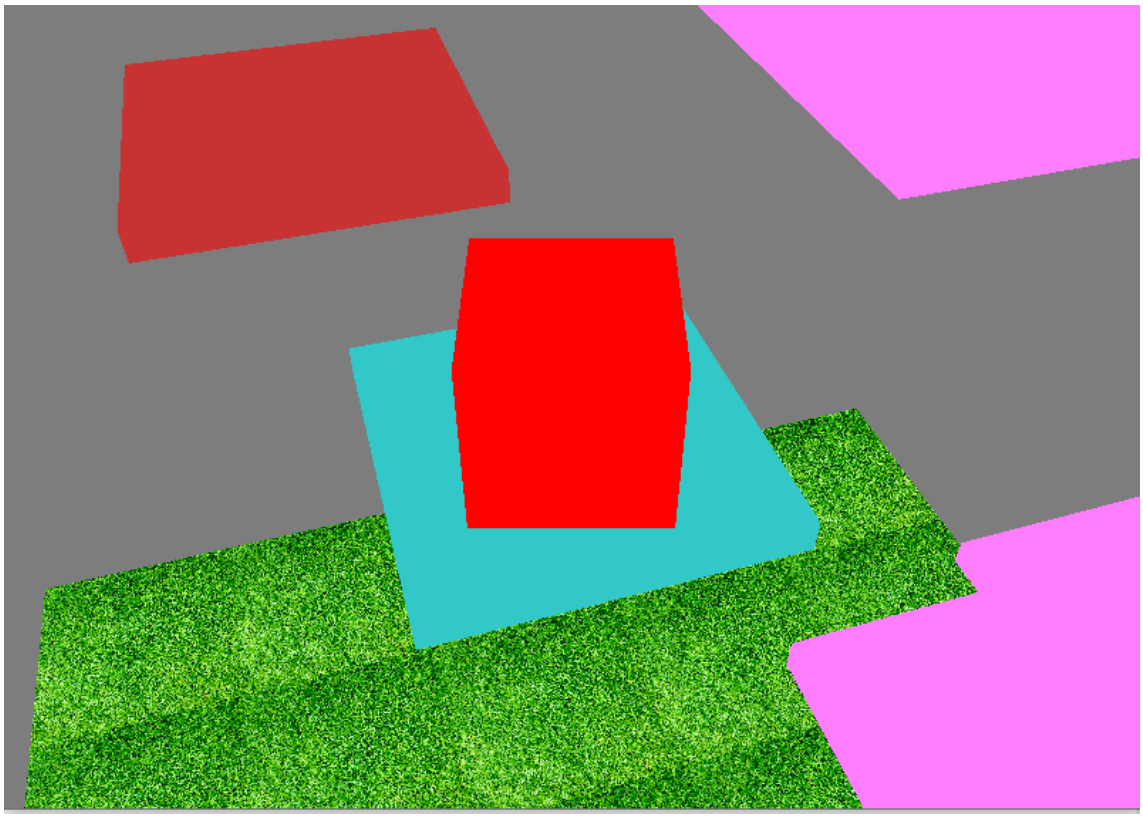
This draws the number of resets left in the top left hand corner of the screen to make the player sure how many they have left.

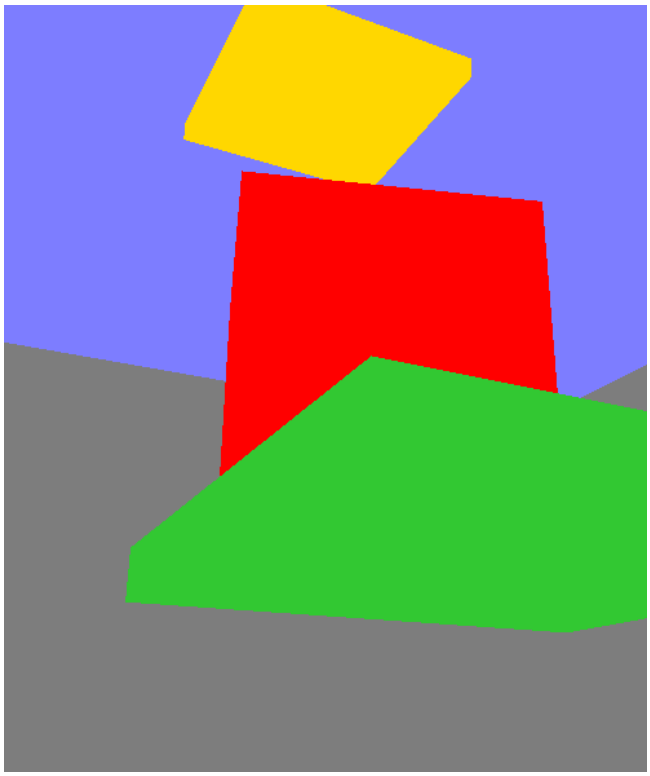
Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework





Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework





Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

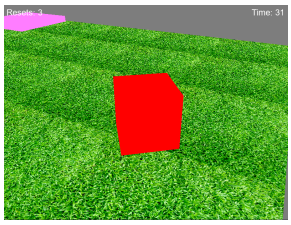
Test 6

test	code	expected	Result

Trap tile test	if platform.get('trap') and platform_pos in trap_timers:
	if current_time - trap_timers[platform_pos] > TRAP_DURATION:
	continue
		When the time reaches the trap duration time it will disspaear	Trap disappears after a set amount of time stood on it

Boost tile test	if platform.get('boost') and keys[K_SPACE]: player_velocity[1] = BOOST_POWER is_jumping = True	When the player reaches this tile and space is pressed the player should jump higher than before	Again this works and the player can jump higher after the boost pad is reached for one jump. Sometimes doesn't activate straight away
Checkpoint tile test	if platform.get('checkpoint'): current_checkpoint = platform['pos'].copy() current_checkpoint[1] += platform['size'][1] + collision_offset	Will get a copy of the pos of the tile when the player touches the tile and saves this for when the player resets as a base to restore the player character slightly above the checkpoint to not mess with collisions	This works and whenever the player clicks the reset button the player restores to the middle of the checkpoint tile
Finish tile check	if platform.get('finish'): LEVEL_COMPLET	The player touches the tile and the finish functions start to register the time and player name	The player gets filler text as the finish function isn't set up yet but the time is being saved

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

test	code	expected	Result
	ED = True is_paused = True		
The Camera	def handle_camera():	The player will follow the camera around as the player moves the camera in a 360 degree motion on the horizontal axis and 180 degree on the vertical axis centred around the player	This is achieved perfectly and follows the expected results
The wall	def draw_walls():	4 Big gray walls spawn which negated the issue of leaving the map the players talk about before and the camera should collide with them	

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Phase Four: draw_2d_text

For this version of the game to make sure I didn't cause any stack overflow, I created a function that uses the ideas from before to create text on a 3d plane 2d.

```
627 def draw_2d_text(surfaces, positions, sizes):
628     """Helper function to draw 2D text with proper matrix handling"""
629     # Save the current matrices
630     glMatrixMode(GL_PROJECTION)
631     glPushMatrix()
632     glLoadIdentity()
633     glOrtho(0, WINDOW_WIDTH, WINDOW_HEIGHT, 0, -1, 1)
634 
635     glMatrixMode(GL_MODELVIEW)
636     glPushMatrix()
637     glLoadIdentity()
638 
639     # Disable depth testing temporarily
640     glDisable(GL_DEPTH_TEST)
641 
642     # Draw all surfaces
643     for surface, pos, size in zip(surfaces, positions, sizes):
644         # Convert surface to texture
645         text_data = pygame.image.tostring(surface, "RGBA", True)
646         text_texture = glGenTextures(1)
647 
648         glBindTexture(GL_TEXTURE_2D, text_texture)
649         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
650         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
651         glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, size[0], size[1], 0, GL_RGBA, GL_UNSIGNED_BYTE, text_data)
652 
653         # Draw the texture
654         glEnable(GL_TEXTURE_2D)
655         glEnable(GL_BLEND)
656         glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
657 
658         glColor4f(1.0, 1.0, 1.0, 1.0)
659         glBegin(GL_QUADS)
660         glTexCoord2f(0, 1); glVertex2f(pos[0], pos[1])
661         glTexCoord2f(1, 1); glVertex2f(pos[0] + size[0], pos[1])
662         glTexCoord2f(1, 0); glVertex2f(pos[0] + size[0], pos[1] + size[1])
663         glTexCoord2f(0, 0); glVertex2f(pos[0], pos[1] + size[1])
664         glEnd()
```

```
666         glDeleteTextures([text_texture])
667 
668     # Restore state
669     glDisable(GL_BLEND)
670     glDisable(GL_TEXTURE_2D)
671     glEnable(GL_DEPTH_TEST)
```

This function takes the ability to load a 2d bit of text on a 3d screen and stores it all in a repeatable function that can then be used again and again whenever I want to generate something on my game.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Phase Five Part One: The finish

Now there is a finish tile that ends the run and timer for the player, there needs to be a completion screen to record the times that will be saved in the database and showing your own time.

```
502 def draw_completion_screen():
503     global elapsed_time, SCORE_SAVED
504     if not SCORE_SAVED:
505         save_score(elapsed_time)
506         SCORE_SAVED = True
507 
508     # Get top scores
509     top_scores = get_top_scores()
510 
511     # Create all text surfaces
512     surfaces = []
513     positions = []
514     sizes = []
515 
516     font = pygame.font.SysFont('Arial', 48)
517     scores_font = pygame.font.SysFont('Arial', 36)
518 
519     # Add completion text
520     completion_surface = font.render("Level Complete!", True, (0, 0, 0))
521     time_surface = font.render(f"Your Time: {int(elapsed_time)} seconds", True, (0, 0, 0))
522     high_scores_surface = font.render("Top Times:", True, (0, 0, 0))
523 
524     # Calculate total height
525     total_height = completion_surface.get_height() + time_surface.get_height() + high_scores_surface.get_height()
526     total_height += len(top_scores) * scores_font.get_height() + (len(top_scores) + 3) * 10  # Adding spacing
527 
528     current_y = (WINDOW_HEIGHT - total_height) // 2
529 
530     # Add main surfaces
531     for surface in [completion_surface, time_surface, high_scores_surface]:
532         x = (WINDOW_WIDTH - surface.get_width()) // 2
533         surfaces.append(surface)
534         positions.append((x, current_y))
535         sizes.append((surface.get_width(), surface.get_height()))
536         current_y += surface.get_height() + 10
537 
```

```
538     # Add score surfaces
539     for i, (name, time, date) in enumerate(top_scores, 1):
540         score_surface = scores_font.render(f"{i}. {name}: {int(time)}s", True, (0, 0, 0))
541         x = (WINDOW_WIDTH - score_surface.get_width()) // 2
542         surfaces.append(score_surface)
543         positions.append((x, current_y))
544         sizes.append((score_surface.get_width(), score_surface.get_height()))
545         current_y += score_surface.get_height() + 10
546 
547     draw_2d_text(surfaces, positions, sizes)
```

This code starts by checking if the current run has been written into the database and if it hasn't it does that.

Then it gets the top 5 scores from the database ready to be displayed.

Then the text for the completion screen is made, saying level completed and the time the player achieved, then under this the top times are rendered.

Now for rendering the top scores for the game it calculates how many it has grabbed and then spaces them out evenly across the screen.

The next two sections are for rendering the scores and completion message on the screen when the finish tile is stepped on.

Phase Five Part Two: The Database

To track the top five scores and track them to a player, I decided to use a database to save the score and their username. I then output the top 5 fastest times to be displayed on the completion screen.

```
559 def init_database():
560     """Initialize the database and create the high scores table if it doesn't exist."""
561     conn = sqlite3.connect('game_scores_project_ver.db')
562     c = conn.cursor()
563     c.execute('''CREATE TABLE IF NOT EXISTS high_scores
564                  (id INTEGER PRIMARY KEY AUTOINCREMENT,
565                   player_name TEXT,
566                   completion_time REAL,
567                   date TEXT)''')
568     conn.commit()
569     conn.close()
570 
571 def save_score(completion_time):
572     """Save the score to the database."""
573     player_name = get_username()
574     current_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
575 
576     conn = sqlite3.connect('game_scores.db')
577     c = conn.cursor()
578     c.execute("INSERT INTO high_scores (player_name, completion_time, date) VALUES (?, ?, ?)",
579               (player_name, completion_time, current_date))
580     conn.commit()
581     conn.close()
582 
583 def get_top_scores(limit=5):
584     """Retrieve the top scores from the database."""
585     conn = sqlite3.connect('game_scores.db')
586     c = conn.cursor()
587     c.execute("SELECT player_name, completion_time, date FROM high_scores ORDER BY completion_time ASC LIMIT ?", (limit,))
588     scores = c.fetchall()
589     conn.close()
590     return scores
591 
592 def reset_database():
593     """Reset the high scores database by deleting all records."""
594     try:
595         conn = sqlite3.connect('game_scores.db')
596         c = conn.cursor()
597         c.execute("DELETE FROM high_scores")  # Delete all records
598         conn.commit()
599         conn.close()
600         print("Database reset successfully")
601     except sqlite3.Error as e:
602         print(f"Error resetting database: {e}")
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Firstly, using the sqlite3 import for python, I make a new database. If it doesn't exist already, it tracks the username, time and the date (date?) the time happened on, storing and creating the database in the same file location as the users code.

The save score function then gets the username the user has inputted and the current time when the player stands on the platform to finish and then connects to the database. Then it inputs these values into the database to then close it back up.

Get top scores then connects to the database and filter the times in order of the fastest completion time and then stores them in the variable scores, so they can be shown by the completion screen at the end.

If the database needs to be cleared as a problem has occurred then by pressing ctrl r, the database will be wiped clean and the players can then input new times again.

Then the username needs to be obtained.

```
604 def get_username():
605     username = ""
606     input_active = True
607     font = pygame.font.SysFont('Arial', 32)
608 
609     while input_active:
610         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
611 
612         for event in pygame.event.get():
613             if event.type == pygame.KEYDOWN:
614                 if event.key == pygame.K_RETURN:
615                     input_active = False
616                 elif event.key == pygame.K_BACKSPACE:
617                     username = username[:-1]
618                 else:
619                     if len(username) < 20:
620                         username += event.unicode
621 
622         prompt_text = font.render("Enter Username:", True, (0, 0, 0))
623         input_text = font.render(username + "|", True, (0, 0, 0))
624 
625         surfaces = [prompt_text, input_text]
626         positions = [
627             ((WINDOW_WIDTH - prompt_text.get_width()) // 2, WINDOW_HEIGHT // 2 - 25),
628             ((WINDOW_WIDTH - input_text.get_width()) // 2, WINDOW_HEIGHT // 2 + 25)
629         ]
630         sizes = [(s.get_width(), s.get_height()) for s in surfaces]
631 
632         draw_2d_text(surfaces, positions, sizes)
633         pygame.display.flip()
634 
635     return username if username else "Player"
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

This code works together to display a prompt on the users screen asking them to input their username, it then stores the username in a variable called username and that then can be stored in the database.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Phase 6: handle_events

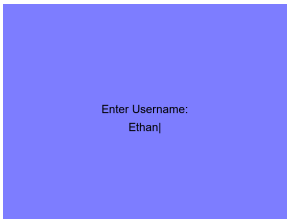
There are many keybinds in the game apart from the basic movement ones, so this is where I will store them and organise them properly

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
689 def handle_events():
690     global is_paused, start_time, stored_time, elapsed_time, LEVEL_COMPLETED, pause_start_time, checkpoint_resets_remaining
691     for event in pygame.event.get():
692         if event.type == pygame.QUIT:
693             pygame.quit()
694             quit()
695         if event.type == pygame.VIDEORESIZE:
696             resize_window(event.w, event.h)
697         if event.type == pygame.KEYDOWN:
698             if event.key == K_ESCAPE and not LEVEL_COMPLETED:
699                 is_paused = not is_paused
700                 if is_paused:
701                     pause_start_time = time.time()
702                     stored_time = elapsed_time
703                     pygame.mouse.set_visible(True)
704                     pygame.event.set_grab(False)
705                 else:
706                     pause_duration = time.time() - pause_start_time
707                     start_time = start_time + pause_duration
708                     pygame.mouse.set_visible(False)
709                     pygame.event.set_grab(True)
710 
711             if event.key == K_BACKSPACE and checkpoint_resets_remaining > 0:
712                 reset_to_checkpoint()
713 
714             if event.key == K_q and is_paused:
715                 pygame.quit()
716                 quit()
```

This checks for the quit keys, reset key and the pause key to alter the state of the game to match what happens, this also pauses the timer by storing the time when paused was pressed.

TEST 7

test	code	expected	Result
Username	def get_username():	A screen when the player reaches the finish tile appears to allow them to enter a username	
Saving score	def save_score(completion_time):	Should see if that username exists and then replace old score if needs be	Multiple of the same name can be used so there can be multiple Ethan/Sarah's

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Display top scores	def get_top_scores(limit=5):	Will look at the database and get the top scores within that and display the top 5	

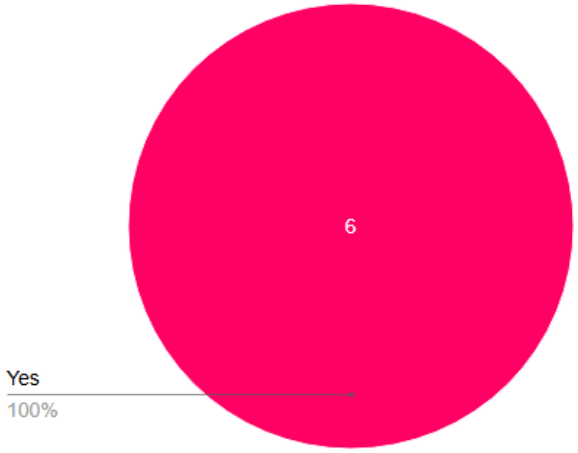
Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Evaluation section

Final testing and success criteria

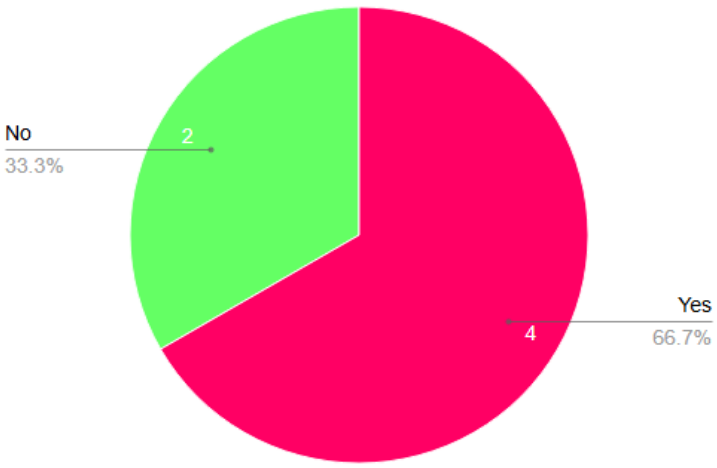
Now that the implementation section of the coursework has been completed, I now need to complete the post development testing. To do this I have gathered data from my stakeholders to get feedback on their game. I did this by sending them a copy of the game along with a survey to fill out

1) Are you happy with the program?



I'm happy to say that all the stakeholders who participated in this test of the final version are happy with how the game turned out.

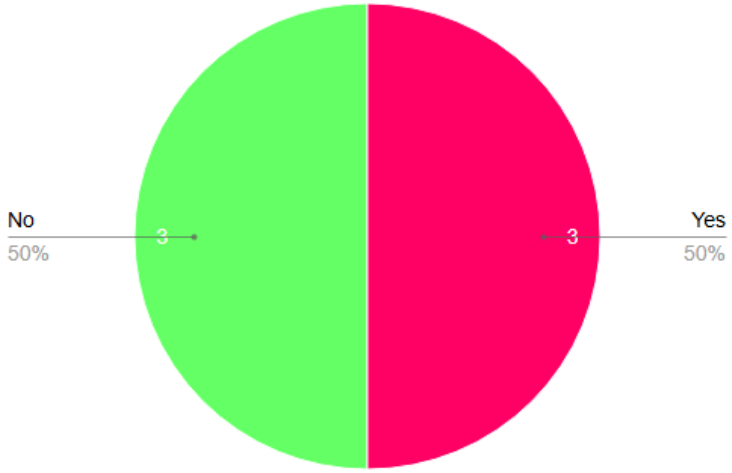
2) Does the Program operate the way you expected?



Most of my stakeholders thought the game ran as they expected it to, but two of them weren't expecting for the game to be boxed into the small tower. *'I wanted to have to be able to explore the area more.'* and *'There weren't hidden areas thanks to the design of the tower.'*

So there was meant to be a more open area to the game, where they could explore and find hidden areas. This became an issue as there were complaints about the boundary of the ground. To fix this I made the game centered around the tower and stopped the issue. But this meant I ended up making some people unhappy. But they must have been happy overall as they still said they were happy with the program.

3) Do you think that the level had a appropriate level of difficulty?

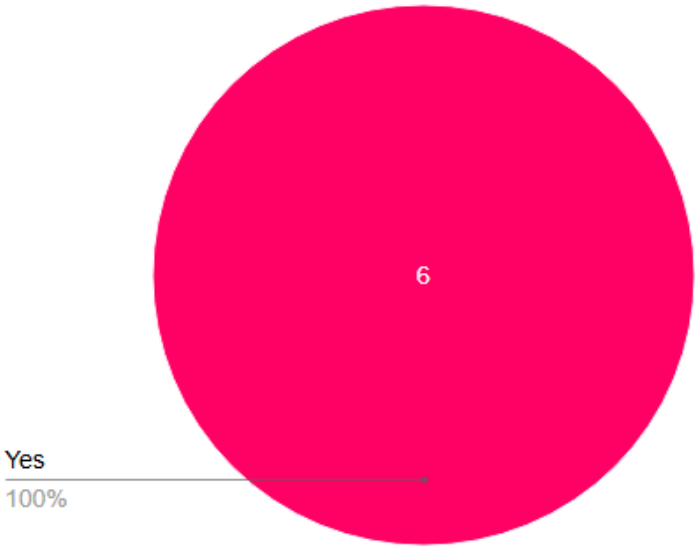


This question was answered at a 50/50. So some thought the game was difficult enough while 3 of them either thought it was too difficult or too easy

'I struggled to be able to make it up the levels as sometimes the bounce pad didn't send me right up, making it too hard to reach the next platform'
'The level was designed to be easy and after a few runs were able to complete it in little time'
'Sometimes the jumps felt impossible thanks to the disappearing platforms, that existed disappearing too fast'

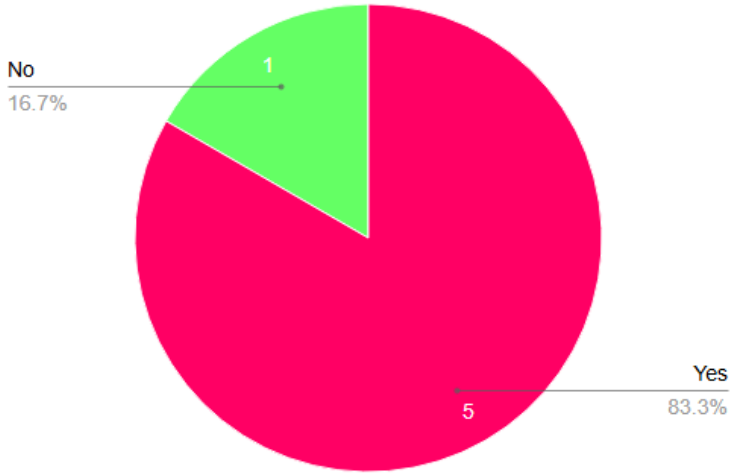
This can be seen that some people had issues, while 4 people either completed the game and felt it was fair or too fas,t while only two struggled. This shows to me that the game was made to a fair level and the struggle was mostly subjective.

4) Were the Different types of platforms good?



I got a totally positive response to this question, as they thought the current range of platforms were good for the first level.
I did get the comment *'in future levels I would like to see more types introduced'* So while more were wanted, they were happy with the current choice for now.

5) Was the program easy to use and understand?



I got a positive response from the stakeholders on if the game was easy to use and understand, this is probably thanks to them being gamers themselves and I made sure to follow standards used by other games as much as possible.

However the one person who said no was because they weren't sure on the quit button. *'I wasn't sure what button to press when I was meant to quit the game, as it wasn't said'.*

I chose 'Q' to be the quit button because it is the letter that the word quit starts with so it made the most logical sense and the other players didn't see it as an issue.

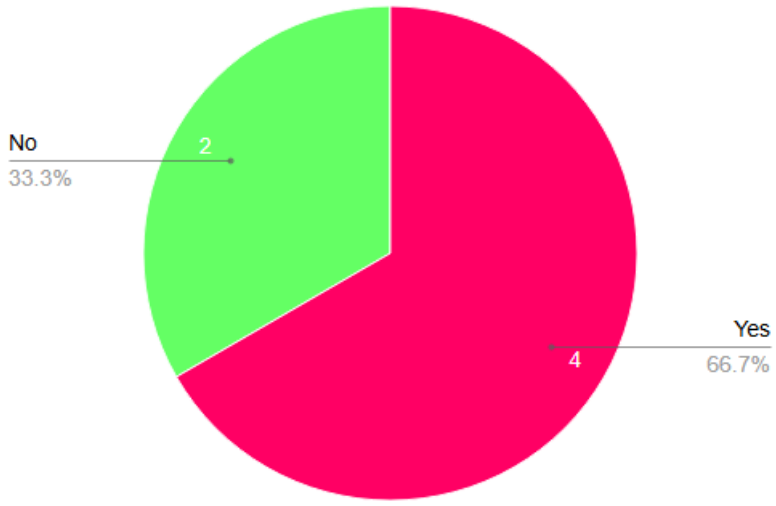
I have then asked the stakeholders on how intuitive they thought the program was and then asked them to give the program a score out of 5. I got a mean rating of 4.3 recurring and I have the reasons below.

Score	Feedback
5	The controls were easy to understand and I had no problems understanding what to do in the game
4	I got a grasp of the controls early on but I wish I had a quit button or instead the pause menu telling you the button to quit
5	I had no problems, the controls and level was easy to understand
3	I had issues playing on my chromebook as when I tried to use my mouse it didn't let me move the player around making the game harder to play
4	The controls were easy to use but there were problems with the boost pads not always working
5	Controls were easy and I understood what to do straight away. Fun Game

I was going to leave it there but since some bugs were reported I figured I had to ask about that.

6) Did you find any bugs in the game?

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework



I have gathered the response from the 4 people who said yes.

I had issues using the jump pad as sometimes they didn't activate straight away
The mouse sometimes didn't work on the chromebook I was using, meaning there was some compatibility issues.
Jump pads sometimes took more than one attempt to use, meaning it slowed me down
Jump pads didn't work all the time

The Chromebook issue was a compatibility between my code and the linux engine that the chromebook was using so I would have to look into that in the future.

But the jump pad issue can be identified as this bit of code which uses the BOOST_POWER bit of code to edit the jump power of the player. It is expected that when the player lands on that platform and jumps it should send them up straight away.

But in some instances it wasn't and that has been identified as when they were jumping straight away. Even though they were classed on a form of ground so the jump was reset, the boost power sometimes wasn't being applied in time. Meaning they had a wasted jump.

This was detrimental to the people who were trying to speedrun each level.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Final Test

Test	Function/ Procdedure	Input	Result
Load Program	def main():	Click to run	Game loads in a 800x600 screen ratio and a the game starts instantly
Exit program	def handle_events( ):	When paused “Q”	cancel the program and shut it down
Start Button pressed	N/A		I never got round to implementing this feature so there is no result or input
Pause menu	def pause_menu():	Esc pressed	This will bring up the menu that can allow the user to exit the game, and stop the timer
World generated	def load_texture(filename): def draw_ground(): def draw_platforms() def draw_walls():	When the game loads through the main function	When the game loads the current first and only level is generated
Character generation	def handle_camera(): def handle_movement(): def draw_player():	When the game loads through main function	The character will then be initialised from the function draw_player() and should allow a camera attached to the character to be moved
Look up	def handle_camera():	Move mouse up	The characters camera should move to look upwards until it reaches the designated max angle

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Test	Function/ Procdedure	Input	Result
Look Down	def handle_camera ():	Move mouse down	The characters camera should move to look downwards until it reaches the designated max angle
Walk forward	def handle_movement():	W key on keyboard pressed	Moves the player forward at constant rate
Walk Left	def handle_movement():	A key on keyboard pressed	Move the player left at a constant rate
Walk Backwards	def handle_movement():	S key on keyboard pressed	Move the player backwards at a constant rate
Walk Right	def handle_movement():	D key on keyboard pressed	Move the player right at a constant rate
Jump	def handle_movement():	The space bar on keyboard is pressed	Cause the player to move upwards Could be vertically or follow the direction of movement key pressed
Double Jump	N/A		I never got round to implementing this feature so there is no result or input
Wall collisions	def handle_movement(): def draw_walls():	Player and wall collide	When pressing the W,A,S,D or jump when the player collides with a wall will it stop
Platform collisions	def handle_movement(): def draw_platforms ():	Player and platforms collide	When pressing the W,A,S,D or jump when the player collides with a platform will it stop

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Test	Function/ Procdedure	Input	Result
Check Point	def draw_platforms (): def reset_to_check point():	A movement key pressed to reach the tile	This draws the checkpoint tiles in designated areas that can be reacher through platforming then saving the When the player resets spawns them back there through saving the X,Y,Z of the platform
Player death	def draw_resets_re maining():	When “BackSpace” is pressed	Resets the player to latest checkpoint, acts like death. Only 3 lives given
Timer save	def save_score(co mpletion_time): def get_top_scores (limit=5):	after going over win tile with W,A,S,D	At the end of a level the timer saves the data to a database Then leaderboard shows
Portal check	N/A		I never got round to implementing this feature so there is no result or input
Traps	def draw_platforms ()	W,A,S,D pressed until on tile	The trap tiles last for a specified amount of time before disappearing dropping the player
Item	N/A		I never got round to implementing this feature so there is no result or input
Unmapped key pressed	def handle_events( ):	A key not mapped is pressed	This does not affect anything to do with game and when pressing any of the keys nothing does happen and the game will continue to run

Now I will then compare the end product to the success criteria to see what I managed to meet and achieve.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

#	Criteria	Success (y/n)	Proof	Comments
1	The game will be fullscreen	n	Iteration 2	I had to change this idea as it was stretching the game to be fullscreen causing the slower version that existed in interaction 1 but was full screen
2	The timer will only start when the player starts to move	y/n	Test 3	The game starts instantly and the player is moving so it is met but because the game starts instantly this is a bit of a way around this
3	The timer does not stop when the player dies	y	Test	
4	A character who can move in all directions - up, down, left, right and diagonally.	y	Test 5	
5	There will be a leaderboard that can track the times recorded of the player that can be compared to other players.	y	Test 6	

#	Criteria	Success (y/n)	Proof	Comments
6	The game will have a multitude of levels to play on	n	Iteration 3	This is because of time constraints meaning there wasn't enough time to implement a new level.
7	Puzzles and traps on each level	y/n	Iteration 3	This is achieved when looking at the traps as they have disappearing tiles, but puzzles never got implemented.
8	Each level will have a different theme	n	Iteration 3	This never got the chance to be implemented as they was only one level
9	Tools and items that can be used to navigate each world	n	Iteration 3	The tools were too complex to try to implement at the moment and I was under strict time constraints making it harder to achieve this.
10	There will be two timers one per level and one for all levels combined	n	Test 3	I only ever made one timer as that was what was needed at the time and didn't need another as there was only one level.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

Usability features

The game didn't really have many game breaking issues when it came to usability apart from the compatibility issue with one of my stakeholders on a chromebook. Leading to the bug for the mouse. It meant the mouse position was resetting back to the centre point making the game hard.

But the other usability features of the game were understood well as they followed the general gaming conventions most people use in their games. Apart from the quit button being q. As this wasn't explained it raised a few complaints but people caught on quite fast to the button change. But I thought it would fit as it is the first letter of quit.

There were some complaints about how the boost pads worked but they were mostly small issues as they were holding space when getting there and can be solved easily in later versions.

Maintenance

I think my final program has been commented on well enough for someone to understand if they decided to look at the code. But they would need this documented to get a full understanding of this program.
There could be the issue that all nearly 800 lines of code are on one python document, making it harder to navigate between each section of the code. But this was done so it was easier for my stakeholders to prepare the game on their own devices and not have problems installing everything.

Limitations and Improvements

All the success criteria that weren't meant, were mostly because they weren't actually implemented due to time constraints or me forgetting to implement them. But ones like having multiple levels means i would need to setup multiple sections of code for different levels and I didn't reach that stage in iterations yet.

If I was to look at improvements, the main and obvious one was to add a new level into the game as that would improve the experience of the people playing the game allowing them to not just have to repeat the same level over and over again.

Another one, would be a full reset button, this would allow the player to not have to close and restart the game every time they wanted to do a rerun of the level, allowing for a smoother game experience.

The last improvement I would make for the final version, would be to change the timer to show milliseconds again to allow the players who get similar times within the same second gap to see who was faster.

But this can't be done now as I don't have time to go back and fix and improve the game anymore as I am approaching my exams for Computer Science and the rest of my A-Levels are coming up. It's also possible that when changing the code, more issues could arise, and I wanted to ensure that the game could be played in my completed submission.

Reflection

By completing the development lifecycle it taught me how to do thorough research and analysis of other works and finding solutions for my idea. It helped me design a rough draft of my game and understand the fine workings needed to make my game work the way i intended. The Implementation section provided quite the challenge of understanding how to code in PyOpenGl and how to create a playable 3d plain. Using Object Orintated Coding to originally create the player and camera class was a good idea but then i struggled to expand that in a way i wanted. So that led to me changing away from this and instead just using multiple linked functions to create the player and camera. As i progressed i learned more about level and game design and how to create a fun and playable game for the end users.

Multiple times when testing there were issues of it nor giving the expected results for failing or even passing the test which led me to have to search over all lines of code over to find any reason for this. Which showed me the importance of proofreading my code to find small mistakes

But as a overall this has taught me well how to document my code and even taught me in a more in depth way about my code and how coding in python works overall.

This comes together to help me find out what I would do differently if i was to do this all again. I found that I was quite overly ambitious when making this game with the idea and concept, It was very ambitious for where i was at with my level coding at the time. I had to rush to learn the methods of OpenGL and how to use pygame. Like switching

between the 2d and 3d plain for text. This took a long time and made it harder for me to complete the game as I had a lot to learn.

But i also didn't do nearly enough research in my Analysis section as when i was developing my success criteria. This shows in the Evaulation section as many of the success criteria were not met and kind of ignored. This was because of my over ambition but also because they didn't fit the needs of the game as it developed. Like any success criteria linked to multiple levels as there was only one ever made.

But then I also made some good progress with my code and used the knowledge i had developed to create a reusable section of the code that would allow me to effectively draw text on screen. This saved me a lot of time and memory for the game. This shows how i developed a better understanding of reusable programming.

There was also the way the platforms were generated. I was happy with how i made this in the end. As it could easily create new platforms that were tailored for a specfric purpose like the boost pads or checkpoints. This made level design easier for the game as the values could be changed in the code on one line without effecting the rest of the platforms position or type.

Overall, I am very happy with where the project has ended up and happy with what I've done. I will probably make changes and further iterations in the future to challenge myself, to expand my coding and testing experience. I also want to see where I can take the game. There are probably some bugs that exist as I may not have asked the right questions and not gathered enough information during the analysis stage of the project.

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

My Code

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
import time
import math4
import sqlite3
from datetime import datetime

# Constants
WINDOW_WIDTH = 1536
WINDOW_HEIGHT = 864
PLAYER_START_POS = [0, 2, 0]
GROUND_SIZE = 7
GRAVITY = -0.02
PLAYER_SPEED = 0.1
JUMP_POWER = 0.2
BOOST_POWER = 0.4
FINISH_COLOR = (1.0, 0.843, 0.0)  # Gold color for finish platform
LEVEL_COMPLETED = False  # Track if level is completed
MAX_CHECKPOINT_RESETS = 3
PLATFORMS = [
    {'pos': [2, 1, -2], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False},    # First platform
    {'pos': [-2, 2, -3], 'size': [2, 0.2, 2], 'trap': True, 'boost': False, 'checkpoint': False},    # Trap platform
    {'pos': [2, 3, -4], 'size': [2, 0.2, 2], 'trap': False, 'boost': True, 'checkpoint': False},     # Boost platform
    {'pos': [2, 7, -3], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False},    # Fourth platform
    {'pos': [3, 8, -1], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False},    # Fifth platform
    {'pos': [1, 9, 1], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': True},   # Checkpoint platform
    {'pos': [-1, 10, 2], 'size': [2, 0.2, 2], 'trap': True, 'boost': False, 'checkpoint': False},    # Trap platform
    {'pos': [1, 11, -1], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False},    # Sixth platform
    {'pos': [-2, 12, -1], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False},    # Seventh platform
    {'pos': [-4, 13, 2], 'size': [2, 0.2, 2], 'trap': False, 'boost': True, 'checkpoint': False},     # Boost platform
    {'pos': [0, 16, 3], 'size': [2, 0.2, 2], 'trap': True, 'boost': False, 'checkpoint': False},    # Trap platform
    {'pos': [-2, 17, 0], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': True},   # Checkpoint platform
    {'pos': [-5, 18, 1], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False},    # Eighth platform
    {'pos': [-3, 19, 2], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False},    # Ninth platform
    {'pos': [-1, 20, 4], 'size': [2, 0.2, 2], 'trap': False, 'boost': True, 'checkpoint': False},     # Boost platform
    {'pos': [-1, 24, 3], 'size': [2, 0.2, 2], 'trap': True, 'boost': False, 'checkpoint': False},    # Trap platform
    {'pos': [0, 25, 0], 'size': [2, 0.2, 2], 'trap': False, 'boost': False, 'checkpoint': False, 'finish': True},  # Finish platform
]
SCORE_SAVED = False

# Setup pygame and OpenGL
pygame.init()
display = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT), DOUBLEBUF | OPENGL)
glEnable(GL_DEPTH_TEST)
glEnable(GL_TEXTURE_2D)
glEnable(GL_BLEND)
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
glClearColor(0.5, 0.5, 1.0, 1.0)  # This is your blue background
glMatrixMode(GL_PROJECTION)
glLoadIdentity()
gluPerspective(45, (WINDOW_WIDTH / WINDOW_HEIGHT), 0.1, 50.0)
glMatrixMode(GL_MODELVIEW)
glLoadIdentity()
glTranslatef(0.0, 0.0, -5)

# Set up movement and physics
player_pos = [0, 2, 0]  # Start position of the player
player_velocity = [0, 0, 0]
player_speed = 0.1
jump_power = 0.2
gravity = -0.02
is_jumping = False
is_paused = False
start_time = time.time()
elapsed_time = 0
stored_time = 0
pause_start_time = 0

# Ground settings
ground_texture = None

#trap settings
trap_timers = {}  # Dictionary to store trap activation times
TRAP_DURATION = 1.5  # Seconds before trap disappears

#camera settings
camera_angle_x = 0  # Camera rotation around Y axis (left/right)
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
camera_angle_y = 0  # Camera rotation around X axis (up/down)
camera_sensitivity = 0.003  # Adjust this to change mouse sensitivity
player_rotation = 0 # Track player cube rotation

# Checkpoint settings
CHECKPOINT_COLOR = (0.2, 0.8, 0.8)  # Cyan-ish color for checkpoint platforms
current_checkpoint = None  # Store the last touched checkpoint position
checkpoint_resets_remaining = MAX_CHECKPOINT_RESETS


def load_texture(filename):
    try:
        # Print current working directory to help with debugging
        import os
        print(f"Current working directory: {os.getcwd()}")
        print(f"Attempting to load texture: {filename}")

        texture = glGenTextures(1)
        img = pygame.image.load(filename)

        # Print texture information
        print(f"Image size: {img.get_size()}")
        print(f"Image format: {img.get_bitsize()}-bit")

        img = pygame.transform.flip(img, False, True)
        img_data = pygame.image.tostring(img, "RGBA", 1)

        glBindTexture(GL_TEXTURE_2D, texture)
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, img.get_width(), img.get_height(),
                     0, GL_RGBA, GL_UNSIGNED_BYTE, img_data)

        # Set texture parameters
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)

        print(f"Successfully loaded texture: {texture}")
        return texture
```

```
    except pygame.error as e:
        print(f"Couldn't load texture {filename}: {e}")
        print("Make sure the image file exists and is in the correct format")
        return None
    except Exception as e:
        print(f"Unexpected error loading texture: {e}")
        return None

def draw_ground():
    if ground_texture is None:
        # Fallback color if texture isn't loaded
        glDisable(GL_TEXTURE_2D)
        glColor3f(0.0, 0.8, 0.0)  # Green ground
    else:
        glEnable(GL_TEXTURE_2D)
        glColor3f(1.0, 1.0, 1.0)  # White color to not tint texture
        glBindTexture(GL_TEXTURE_2D, ground_texture)

    glBegin(GL_QUADS)
    # Ground plane with texture coordinates
    glTexCoord2f(0.0, 0.0)
    glVertex3f(-7, 0, 7)  # Near left
    glTexCoord2f(5.0, 0.0)
    glVertex3f(7, 0, 7)   # Near right
    glTexCoord2f(5.0, 5.0)
    glVertex3f(7, 0, -7)  # Far right
    glTexCoord2f(0.0, 5.0)
    glVertex3f(-7, 0, -7) # Far left
    glEnd()

    glDisable(GL_TEXTURE_2D)

def draw_platforms():
    current_time = time.time()
    for platform in PLATFORMS:
        platform_pos = tuple(platform['pos'])  # Convert list to tuple
        # Skip drawing if it's a triggered trap that has expired
        if platform.get('trap') and platform_pos in trap_timers:
            if current_time - trap_timers[platform_pos] > TRAP_DURATION:
                continue
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
        glPushMatrix()
        glTranslatef(*platform['pos'])
        glScalef(*platform['size'])
        glBegin(GL_QUADS)

        # Set color based on platform type
        if platform.get('trap'):
            glColor3f(0.8, 0.2, 0.2)  # Red for trap platforms
        elif platform.get('boost'):
            glColor3f(0.2, 0.8, 0.2)  # Green for boost platforms
        elif platform.get('checkpoint'):
            glColor3f(*CHECKPOINT_COLOR)  # Cyan for checkpoint platforms
        elif platform.get('finish'):
            glColor3f(*FINISH_COLOR)  # Gold for finish platform
        else:
            glColor3f(2.0, 0.5, 1.0)  # Gray for regular platforms

        # Top face
        glVertex3f(-0.5, 0.5, -0.5)
        glVertex3f(0.5, 0.5, -0.5)
        glVertex3f(0.5, 0.5, 0.5)
        glVertex3f(-0.5, 0.5, 0.5)

        # Bottom face
        glVertex3f(-0.5, -0.5, -0.5)
        glVertex3f(0.5, -0.5, -0.5)
        glVertex3f(0.5, -0.5, 0.5)
        glVertex3f(-0.5, -0.5, 0.5)

        # Front face
        glVertex3f(-0.5, -0.5, 0.5)
        glVertex3f(0.5, -0.5, 0.5)
        glVertex3f(0.5, 0.5, 0.5)
        glVertex3f(-0.5, 0.5, 0.5)

        # Back face
        glVertex3f(-0.5, -0.5, -0.5)
        glVertex3f(0.5, -0.5, -0.5)
        glVertex3f(0.5, 0.5, -0.5)
```

```
        glVertex3f(-0.5, 0.5, -0.5)

        # Right face
        glVertex3f(0.5, -0.5, -0.5)
        glVertex3f(0.5, 0.5, -0.5)
        glVertex3f(0.5, 0.5, 0.5)
        glVertex3f(0.5, -0.5, 0.5)

        # Left face
        glVertex3f(-0.5, -0.5, -0.5)
        glVertex3f(-0.5, 0.5, -0.5)
        glVertex3f(-0.5, 0.5, 0.5)
        glVertex3f(-0.5, -0.5, 0.5)

        glEnd()
        glPopMatrix()

def draw_walls():
    wall_height = 25.0  # Height of the walls
    ground_size = 7.0  # Size of the ground (matches your GROUND_SIZE)

    glColor3f(0.5, 0.5, 0.5)  # Gray color for walls
    glBegin(GL_QUADS)

    # North wall
    glVertex3f(-ground_size, 0, -ground_size)
    glVertex3f(ground_size, 0, -ground_size)
    glVertex3f(ground_size, wall_height, -ground_size)
    glVertex3f(-ground_size, wall_height, -ground_size)

    # South wall
    glVertex3f(-ground_size, 0, ground_size)
    glVertex3f(ground_size, 0, ground_size)
    glVertex3f(ground_size, wall_height, ground_size)
    glVertex3f(-ground_size, wall_height, ground_size)

    # East wall
    glVertex3f(ground_size, 0, -ground_size)
    glVertex3f(ground_size, 0, ground_size)
    glVertex3f(ground_size, wall_height, ground_size)
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
    glVertex3f(ground_size, wall_height, -ground_size)

    # West wall
    glVertex3f(-ground_size, 0, -ground_size)
    glVertex3f(-ground_size, 0, ground_size)
    glVertex3f(-ground_size, wall_height, ground_size)
    glVertex3f(-ground_size, wall_height, -ground_size)

    glEnd()

def init_mouse_handling():
    """Initializes mouse handling by hiding the cursor and locking it to the center of the window."""
    pygame.mouse.set_visible(False)  # Hide the mouse cursor
    pygame.mouse.set_pos(WINDOW_WIDTH // 2, WINDOW_HEIGHT // 2)  # Recenter the mouse at the center of the screen

def handle_camera():
    global camera_angle_x, camera_angle_y, player_rotation

    # Get mouse movement
    mouse_rel = pygame.mouse.get_rel()

    # Apply the mouse movement to the camera angles
    camera_angle_x -= mouse_rel[0] * camera_sensitivity
    camera_angle_y -= mouse_rel[1] * camera_sensitivity

    # Limit vertical camera rotation
    camera_angle_y = max(min(camera_angle_y, math.pi / 3), -math.pi / 3)

    # Sync player rotation with camera angle
    player_rotation = camera_angle_x

    # Recenter the mouse after processing the movement to prevent it from leaving the window
    pygame.mouse.set_pos(WINDOW_WIDTH // 2, WINDOW_HEIGHT // 2)

    # Calculate camera position and look direction (unchanged from your original code)
    camera_distance = 5
    dir_x = math.cos(camera_angle_y) * math.sin(camera_angle_x)
    dir_y = math.sin(camera_angle_y)
    dir_z = math.cos(camera_angle_y) * math.cos(camera_angle_x)

```

```
    camera_x = player_pos[0] + camera_distance * dir_x
    camera_y = player_pos[1] + camera_distance * dir_y
    camera_z = player_pos[2] + camera_distance * dir_z

    if camera_y < 0.5:
        camera_y = 0.5

    # Camera collision detection (unchanged from your original code)
    wall_buffer = 0.5
    if camera_x > GROUND_SIZE - wall_buffer:
        camera_x = GROUND_SIZE - wall_buffer
    if camera_x < -GROUND_SIZE + wall_buffer:
        camera_x = -GROUND_SIZE + wall_buffer
    if camera_z > GROUND_SIZE - wall_buffer:
        camera_z = GROUND_SIZE - wall_buffer
    if camera_z < -GROUND_SIZE + wall_buffer:
        camera_z = -GROUND_SIZE + wall_buffer

    # Look at the cube's position
    glLoadIdentity()
    gluLookAt(
        camera_x, camera_y, camera_z,
        player_pos[0], player_pos[1], player_pos[2],
        0, 1, 0
    )

def handle_movement():
    global player_pos, player_velocity, is_jumping, is_paused, current_checkpoint, LEVEL_COMPLETED
    keys = pygame.key.get_pressed()

    if not is_paused:
        # Calculate movement direction based on cube's rotation
        forward_vec = [math.sin(player_rotation), 0, math.cos(player_rotation)]
        right_vec = [math.cos(player_rotation), 0, -math.sin(player_rotation)]

        # Movement Controls always relative to cube's facing
        if keys[K_w]:
            player_pos[0] -= forward_vec[0] * player_speed
            player_pos[2] -= forward_vec[2] * player_speed
        if keys[K_s]:
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
            player_pos[0] += forward_vec[0] * player_speed
            player_pos[2] += forward_vec[2] * player_speed
        if keys[K_a]:
            player_pos[0] -= right_vec[0] * player_speed
            player_pos[2] -= right_vec[2] * player_speed
        if keys[K_d]:
            player_pos[0] += right_vec[0] * player_speed
            player_pos[2] += right_vec[2] * player_speed

        # Jumping
        if not is_jumping and keys[K_SPACE]:
            player_velocity[1] = jump_power
            is_jumping = True

        # Apply gravity
        player_velocity[1] += gravity
        player_pos[1] += player_velocity[1]

        # Check for collisions with ground
        if player_pos[1] <= 0.5:
            player_pos[1] = 0.5
            is_jumping = False
            player_velocity[1] = 0

        # Check for collisions with all platforms
        collision_offset = 0.4
        current_time = time.time()

        for platform in PLATFORMS:
            platform_pos = tuple(platform['pos'])  # Convert list to tuple
            # Skip collision check if it's a triggered trap that has expired
            if platform.get('trap') and platform_pos in trap_timers:
                if current_time - trap_timers[platform_pos] > TRAP_DURATION:
                    continue

            if (
                platform['pos'][0] - platform['size'][0] / 2 - collision_offset <= player_pos[0] <= platform['pos'][0] + platform['size'][0] / 2 + collision_offset and
                platform['pos'][1] - collision_offset <= player_pos[1] <= platform['pos'][1] + platform['size'][1] + collision_offset and
                platform['pos'][2] - platform['size'][2] / 2 - collision_offset <= player_pos[2] <= platform['pos'][2] + platform['size'][2] / 2 + collision_offset
            ):
```

```
                player_pos[1] = platform['pos'][1] + platform['size'][1] + collision_offset
                is_jumping = False
                player_velocity[1] = 0

                # Handle checkpoint platforms
                if platform.get('checkpoint'):
                    current_checkpoint = platform['pos'].copy()
                    current_checkpoint[1] += platform['size'][1] + collision_offset

                # Start timer for trap platforms on first touch
                if platform.get('trap') and platform_pos not in trap_timers:
                    trap_timers[platform_pos] = current_time

                # Handle boost platforms
                if platform.get('boost') and keys[K_SPACE]:
                    player_velocity[1] = BOOST_POWER
                    is_jumping = True

                # Check if it's the finish platform
                if platform.get('finish'):
                    LEVEL_COMPLETED = True
                    is_paused = True

        wall_buffer = 0.5  # Distance from wall to stop player
        if player_pos[0] > GROUND_SIZE - wall_buffer:
            player_pos[0] = GROUND_SIZE - wall_buffer
        if player_pos[0] < -GROUND_SIZE + wall_buffer:
            player_pos[0] = -GROUND_SIZE + wall_buffer
        if player_pos[2] > GROUND_SIZE - wall_buffer:
            player_pos[2] = GROUND_SIZE - wall_buffer
        if player_pos[2] < -GROUND_SIZE + wall_buffer:
            player_pos[2] = -GROUND_SIZE + wall_buffer



def draw_player():
    glPushMatrix()
    glTranslatef(player_pos[0], player_pos[1], player_pos[2])
    glRotatef(math.degrees(player_rotation), 0, 1, 0)
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
    glBegin(GL_QUADS)
    glColor3f(1, 0, 0)  # Red color

    # Front face
    glVertex3f(-0.5, -0.5, -0.5)
    glVertex3f(0.5, -0.5, -0.5)
    glVertex3f(0.5, 0.5, -0.5)
    glVertex3f(-0.5, 0.5, -0.5)

    # Back face
    glVertex3f(-0.5, -0.5, 0.5)
    glVertex3f(0.5, -0.5, 0.5)
    glVertex3f(0.5, 0.5, 0.5)
    glVertex3f(-0.5, 0.5, 0.5)

    # Top face
    glVertex3f(-0.5, 0.5, -0.5)
    glVertex3f(0.5, 0.5, -0.5)
    glVertex3f(0.5, 0.5, 0.5)
    glVertex3f(-0.5, 0.5, 0.5)

    # Bottom face
    glVertex3f(-0.5, -0.5, -0.5)
    glVertex3f(0.5, -0.5, -0.5)
    glVertex3f(0.5, -0.5, 0.5)
    glVertex3f(-0.5, -0.5, 0.5)

    # Right face
    glVertex3f(0.5, -0.5, -0.5)
    glVertex3f(0.5, 0.5, -0.5)
    glVertex3f(0.5, 0.5, 0.5)
    glVertex3f(0.5, -0.5, 0.5)

    # Left face
    glVertex3f(-0.5, -0.5, -0.5)
    glVertex3f(-0.5, 0.5, -0.5)
    glVertex3f(-0.5, 0.5, 0.5)
    glVertex3f(-0.5, -0.5, 0.5)

    glEnd()
```

```
    glPopMatrix()

def reset_to_checkpoint():
    global player_pos, trap_timers, current_checkpoint, SCORE_SAVED, checkpoint_resets_remaining

    if checkpoint_resets_remaining <= 0:
        return  # Don't allow reset if no resets remaining

    # Reset player position to last checkpoint or start position
    if current_checkpoint:
        player_pos = current_checkpoint.copy()
    else:
        player_pos = PLAYER_START_POS.copy()

    # Clear all trap timers to reset traps
    trap_timers.clear()

    # Reset the score saved flag
    SCORE_SAVED = False

    # Decrease remaining resets
    checkpoint_resets_remaining -= 1

def draw_resets_remaining():
    font = pygame.font.SysFont('Arial', 24)
    resets_text = font.render(f"Resets: {checkpoint_resets_remaining}", True, (255, 255, 255))
    resets_surface = pygame.Surface((resets_text.get_width(), resets_text.get_height()), pygame.SRCALPHA)
    resets_surface.blit(resets_text, (0, 0))

    x = 10  # Position on the left side
    y = 10  # Position at the top

    draw_2d_text([resets_surface], [(x, y)], [(resets_surface.get_width(), resets_surface.get_height())])

def draw_timer():
    global elapsed_time
    if not is_paused:
        current_time = time.time()
        elapsed_time = current_time - start_time

```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
    font = pygame.font.SysFont('Arial', 24)
    timer_text = font.render(f"Time: {int(elapsed_time)}", True, (255, 255, 255))
    timer_surface = pygame.Surface((timer_text.get_width(), timer_text.get_height()), pygame.SRCALPHA)
    timer_surface.blit(timer_text, (0, 0))

    x = WINDOW_WIDTH - timer_surface.get_width() - 10
    y = 10

    draw_2d_text([timer_surface], [(x, y)], [(timer_surface.get_width(), timer_surface.get_height())])

def pause_menu():
    font = pygame.font.SysFont('Arial', 36)
    pause_text = font.render("PAUSED", True, (0, 0, 0))
    pause_surface = pygame.Surface((pause_text.get_width(), pause_text.get_height()), pygame.SRCALPHA)
    pause_surface.blit(pause_text, (0, 0))

    x = (WINDOW_WIDTH - pause_surface.get_width()) // 2
    y = (WINDOW_HEIGHT - pause_surface.get_height()) // 2

    draw_2d_text([pause_surface], [(x, y)], [(pause_surface.get_width(), pause_surface.get_height())])

def draw_completion_screen():
    global elapsed_time, SCORE_SAVED
    if not SCORE_SAVED:
        save_score(elapsed_time)
        SCORE_SAVED = True

    # Get top scores
    top_scores = get_top_scores()

    # Create all text surfaces
    surfaces = []
    positions = []
    sizes = []

    font = pygame.font.SysFont('Arial', 48)
    scores_font = pygame.font.SysFont('Arial', 36)

    # Add completion text
    completion_surface = font.render("Level Complete!", True, (0, 0, 0))
```

```
    time_surface = font.render(f"Your Time: {int(elapsed_time)} seconds", True, (0, 0, 0))
    high_scores_surface = font.render("Top Times:", True, (0, 0, 0))

    # Calculate total height
    total_height = completion_surface.get_height() + time_surface.get_height() + high_scores_surface.get_height()
    total_height += len(top_scores) * scores_font.get_height() + (len(top_scores) + 3) * 10  # Adding spacing

    current_y = (WINDOW_HEIGHT - total_height) // 2

    # Add main surfaces
    for surface in [completion_surface, time_surface, high_scores_surface]:
        x = (WINDOW_WIDTH - surface.get_width()) // 2
        surfaces.append(surface)
        positions.append((x, current_y))
        sizes.append((surface.get_width(), surface.get_height()))
        current_y += surface.get_height() + 10

    # Add score surfaces
    for i, (name, time, date) in enumerate(top_scores, 1):
        score_surface = scores_font.render(f"{i}. {name}: {int(time)}s", True, (0, 0, 0))
        x = (WINDOW_WIDTH - score_surface.get_width()) // 2
        surfaces.append(score_surface)
        positions.append((x, current_y))
        sizes.append((score_surface.get_width(), score_surface.get_height()))
        current_y += score_surface.get_height() + 10

    draw_2d_text(surfaces, positions, sizes)

def init_database():
    """Initialize the database and create the high scores table if it doesn't exist."""
    conn = sqlite3.connect('game_scores.db')
    c = conn.cursor()
    c.execute('''CREATE TABLE IF NOT EXISTS high_scores
                 (id INTEGER PRIMARY KEY AUTOINCREMENT,
                  player_name TEXT,
                  completion_time REAL,
                  date TEXT)''')
    conn.commit()
    conn.close()

```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```python
def save_score(completion_time):
    """Save the score to the database."""
    player_name = get_username()
    current_date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    conn = sqlite3.connect('game_scores.db')
    c = conn.cursor()
    c.execute("INSERT INTO high_scores (player_name, completion_time, date) VALUES (?, ?, ?)",
              (player_name, completion_time, current_date))
    conn.commit()
    conn.close()

def get_top_scores(limit=5):
    """Retrieve the top scores from the database."""
    conn = sqlite3.connect('game_scores.db')
    c = conn.cursor()
    c.execute("SELECT player_name, completion_time, date FROM high_scores ORDER BY completion_time ASC LIMIT ?", (limit,))
    scores = c.fetchall()
    conn.close()
    return scores

def reset_database():
    """Reset the high scores database by deleting all records."""
    try:
        conn = sqlite3.connect('game_scores.db')
        c = conn.cursor()
        c.execute("DELETE FROM high_scores")  # Delete all records
        conn.commit()
        conn.close()
        print("Database reset successfully")
    except sqlite3.Error as e:
        print(f"Error resetting database: {e}")

def get_username():
    username = ""
    input_active = True
    font = pygame.font.SysFont('Arial', 32)

    while input_active:
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```python

        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_RETURN:
                    input_active = False
                elif event.key == pygame.K_BACKSPACE:
                    username = username[:-1]
                else:
                    if len(username) < 20:
                        username += event.unicode

        prompt_text = font.render("Enter Username:", True, (0, 0, 0))
        input_text = font.render(username + "|", True, (0, 0, 0))

        surfaces = [prompt_text, input_text]
        positions = [
            ((WINDOW_WIDTH - prompt_text.get_width()) // 2, WINDOW_HEIGHT // 2 - 25),
            ((WINDOW_WIDTH - input_text.get_width()) // 2, WINDOW_HEIGHT // 2 + 25)
        ]
        sizes = [(s.get_width(), s.get_height()) for s in surfaces]

        draw_2d_text(surfaces, positions, sizes)
        pygame.display.flip()

    return username if username else "Player"

def draw_2d_text(surfaces, positions, sizes):
    """Helper function to draw 2D text with proper matrix handling"""
    # Save the current matrices
    glMatrixMode(GL_PROJECTION)
    glPushMatrix()
    glLoadIdentity()
    glOrtho(0, WINDOW_WIDTH, WINDOW_HEIGHT, 0, -1, 1)

    glMatrixMode(GL_MODELVIEW)
    glPushMatrix()
    glLoadIdentity()

    # Disable depth testing temporarily
    glDisable(GL_DEPTH_TEST)
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
    # Draw all surfaces
    for surface, pos, size in zip(surfaces, positions, sizes):
        # Convert surface to texture
        text_data = pygame.image.tostring(surface, "RGBA", True)
        text_texture = glGenTextures(1)

        glBindTexture(GL_TEXTURE_2D, text_texture)
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, size[0], size[1], 0, GL_RGBA, GL_UNSIGNED_BYTE, text_data)

        # Draw the texture
        glEnable(GL_TEXTURE_2D)
        glEnable(GL_BLEND)
        glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)

        glColor4f(1.0, 1.0, 1.0, 1.0)
        glBegin(GL_QUADS)
        glTexCoord2f(0, 1); glVertex2f(pos[0], pos[1])
        glTexCoord2f(1, 1); glVertex2f(pos[0] + size[0], pos[1])
        glTexCoord2f(1, 0); glVertex2f(pos[0] + size[0], pos[1] + size[1])
        glTexCoord2f(0, 0); glVertex2f(pos[0], pos[1] + size[1])
        glEnd()

        glDeleteTextures([text_texture])

    # Restore state
    glDisable(GL_BLEND)
    glDisable(GL_TEXTURE_2D)
    glEnable(GL_DEPTH_TEST)

    # Restore matrices
    glMatrixMode(GL_PROJECTION)
    glPopMatrix()
    glMatrixMode(GL_MODELVIEW)
    glPopMatrix()

def handle_events():
    global is_paused, start_time, stored_time, elapsed_time, LEVEL_COMPLETED, pause_start_time, checkpoint_resets_remaining
```

```
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            quit()
        if event.type == pygame.VIDEORESIZE:
            resize_window(event.w, event.h)
        if event.type == pygame.KEYDOWN:
            if event.key == K_ESCAPE and not LEVEL_COMPLETED:
                is_paused = not is_paused
                if is_paused:
                    pause_start_time = time.time()
                    stored_time = elapsed_time
                    pygame.mouse.set_visible(True)
                    pygame.event.set_grab(False)
                else:
                    pause_duration = time.time() - pause_start_time
                    start_time = start_time + pause_duration
                    pygame.mouse.set_visible(False)
                    pygame.event.set_grab(True)

            if event.key == K_BACKSPACE and checkpoint_resets_remaining > 0:
                reset_to_checkpoint()

            if event.key == K_q and is_paused:
                pygame.quit()
                quit()

            if event.key == K_r and event.mod & pygame.KMOD_CTRL:
                reset_database()
def cleanup():
    global ground_texture
    try:
        if ground_texture is not None and glIsTexture(ground_texture):
            glDeleteTextures([ground_texture])
    except:
        pass  # Ignore any OpenGL errors during cleanup
    finally:
        pygame.quit()

def resize_window(width, height):
```

Ethan Birks
Candidate Number: 3022
A-Level Computer Science Coursework

```
    if height == 0:
        height = 1
    glViewport(0, 0, width, height)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluPerspective(45.0, float(width)/float(height), 0.1, 50.0)
    glMatrixMode(GL_MODELVIEW)

def main():
    global ground_texture, is_paused
    ground_texture = load_texture('grass.jpg')
    clock = pygame.time.Clock()

    # Initialize the database
    init_database()

    init_mouse_handling()

    pygame.mouse.set_visible(False)  # Hide the cursor
    pygame.event.set_grab(True)      # Lock the mouse to the window

    try:
        while True:
            handle_events()

            # Clear the screen
            glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

            if not is_paused:
                glLoadIdentity()  # Reset the modelview matrix
                glTranslatef(0.0, 0.0, -5)  # Move back into the scene
                handle_camera()
                handle_movement()
                draw_ground()
                draw_walls()
                draw_platforms()
                draw_player()
                draw_timer()
                draw_resets_remaining()
            else:
```

```
                draw_ground()
                draw_walls()
                draw_platforms()
                draw_player()
                draw_timer()
                draw_resets_remaining()
                if LEVEL_COMPLETED:
                    draw_completion_screen()
                else:
                    pause_menu()

            pygame.display.flip()
            clock.tick(60)

    except SystemExit:
        cleanup()
        return
    except Exception as e:
        print(f"An error occurred: {e}")
        cleanup()
        return

if __name__ == '__main__':
    main()

```